

MODULES IN ROBINSON SPACES

MIKHAEL CARMONA^{*†}, VICTOR CHEPOI^{*}, GUYS-LAIN NAVES^{*}, AND PASCAL PRÉA^{*†}

Abstract. A *Robinson space* is a dissimilarity space (X, d) (i.e., a set X of size n and a dissimilarity d on X) for which there exists a total order $<$ on X such that $x < y < z$ implies that $d(x, z) \geq \max\{d(x, y), d(y, z)\}$. Recognizing if a dissimilarity space is Robinson has numerous applications in seriation and classification. An *mmodule* of (X, d) (generalizing the notion of a module in graph theory) is a subset M of X which is not distinguishable from the outside of M , i.e., the distance from any point of $X \setminus M$ to all points of M is the same. If p is any point of X , then $\{p\}$ and the maximal by inclusion mmmodules of (X, d) not containing p define a partition of X , called the *copoint partition*. In this paper, we investigate the structure of mmmodules in Robinson spaces and use it and the copoint partition to design a simple and practical divide-and-conquer algorithm for recognition of Robinson spaces in optimal $O(n^2)$ time.

Key words. Robinson dissimilarity, Seriation, Classification, Mmodule, Divide-and-conquer.

AMS subject classifications. 68R01, 05C85, 68P10

1. Introduction. A major issue in classification and data analysis is to visualize simple geometrical and relational structures between objects based on their pairwise distances. Many applied algorithmic problems ranging from archeological dating through DNA sequencing and numerical ecology to sparse matrix reordering and overlapping clustering involve ordering a set of objects so that closely coupled elements are placed near each other. The classical *seriation problem*, introduced by Robinson [35] as a tool to seriate archeological deposits, asks to find a simultaneous ordering (or permutation) of the rows and the columns of the dissimilarity matrix so that small values should be concentrated around the main diagonal as closely as possible, whereas large values should fall as far from it as possible. This goal is best achieved by considering the so-called Robinson property: a dissimilarity matrix has the *Robinson property* if its values increase monotonically in the rows and the columns when moving away from the main diagonal in both directions. A *Robinson matrix* is a dissimilarity matrix which can be transformed by permuting its rows and columns to a matrix having the Robinson property. The permutation which leads to a matrix with the Robinson property is called a *compatible order*. Computing a compatible order can be viewed as the two-dimensional version of the sorting problem. In this paper, we present a simple and practical divide-and-conquer algorithm for computing a compatible order and thus recognizing Robinson matrices in optimal $O(n^2)$ time.

1.1. Related work. Due to the importance in seriation and classification, the algorithmic problem of recognizing Robinson dissimilarities/matrices attracted the interest of many authors and several polynomial time recognition algorithms have been proposed. The existing algorithms can be classified into *combinatorial* and *spectral*. All combinatorial algorithms are based on the correspondence between Robinson dissimilarities and interval hypergraphs. The main difficulty arising in recognition algorithms is the existence of several compatible orders. Historically, the first recognition algorithm was given in 1984 by Mirkin and Rodin [31] and consists in testing if the hypergraph of balls is an interval hypergraph; it runs in $O(n^4)$ time. Chepoi and Fichet [11] gave a simple divide-and-conquer algorithm running in $O(n^3)$ time. The algorithm divides the set of points into subsets and refines the obtained subsets into classes to which the recursion can be applied. Seston [37] presented another $O(n^3)$ time algorithm, by using threshold graphs. In [36], he improved the complexity of his algorithm to $O(n^2 \log n)$. Finally, in 2014 Préa and Fortin [33] presented an algorithm running in optimal $O(n^2)$ time. The efficiency of the algorithm of [33] is due to the use of the PQ-trees of Booth and Lueker [5] as a data structure for encoding all compatible orders. Even if optimal, the algorithm of [33] is far from being simple. Subsequently, two new recognition algorithms were proposed by Laurent and Seminaroti: in [28] they presented an algorithm of complexity $O(\alpha \cdot n)$ based on classical LexBFS traversal and divide-and-conquer (where α is the depth of the recursion tree, which is at most the number of distinct elements of the input matrix), and in [29] they presented an $O(n^2 \log n)$ algorithm, which extends LexBFS to weighted matrices and is used as a multisweep traversal. Laurent, Seminaroti and Tanigawa [30] presented a characterization of Robinson matrices in terms of forbidden substructures, extending the notion of asteroidal

^{*}LIS, Aix-Marseille Université, CNRS, and Université de Toulon, Marseille, France (mikhael.carmona, victor.chepoi, guys-lain.naves, pascal.prea@lis-lab.fr)

[†]École Centrale Marseille, Marseille, France

triples in graphs to weighted graphs. More recently, Aracena and Thraves Caro [1] presented a parametrized algorithm for the NP-complete problem of recognition of Robinson incomplete matrices (*i.e.* determining if an incomplete matrix can be completed into a Robinson one). Armstrong et al. [2] presented an optimal $O(n^2)$ time algorithm for the recognition of strict circular Robinson dissimilarities (Hubert et al. [24] defined circular seriation first and it was studied also in the papers [34] and [26]). A simple and optimal algorithm for strict circular seriation was proposed in [8].

The spectral approach was introduced by Atkins et al. [3] and was subsequently used in numerous papers (see, for example, [20] and the references therein). The method is based on the computation of the second smallest eigenvalue and its eigenvector of the Laplacian of a similarity matrix A , called the *Fiedler value* and the *Fiedler vector* of A . Atkins et al. [3, Theorems 3.2 & 3.3] proved that if A is Robinson, then it has a monotone Fiedler vector and if A is Robinson with a Fiedler value and a Fiedler vector with no repeated values, then the two permutations of the Fiedler vector in which the coordinates are strictly increasing (respectively, decreasing) are the only two compatible orders of A . For similarity matrices for which the Fiedler vector has repeated values, Atkins et al. [3] recursively apply the algorithm to each submatrix of A defined by coordinates of the Fiedler vector with the same value. In this case, they also use PQ-trees to represent the compatible orders. This leads to an algorithm of complexity $O(nT(n) + n^2 \log n)$ to recognize if a similarity matrix is Robinson, where $T(n)$ is the complexity of computing the Fiedler vector of a matrix. The Fiedler vector is computed by the Lanczos algorithm, which is an iterative numerical algorithm that at each iteration performs a multiplication of the input matrix A by a vector.

Real data contain errors, therefore the dissimilarity matrix D can be measured only approximatively and fails to be Robinson. Thus we are lead to the problem of approximating D by a Robinson dissimilarity R . As an error measure one can use the usual ℓ_p -distance $\|D - R\|_p$ between two $n \times n$ matrices. This ℓ_p -fitting problem has been shown to be NP-hard for $p = 1$ [4] and for $p = \infty$ [12]. Various heuristics for this optimization problem have been considered in [6, 23, 25] and papers cited therein. The approximability of this fitting problem for any $1 \leq p < \infty$ is open. Chepoi and Seston [13] presented a factor 16 approximation for the ℓ_∞ -fitting problem. For a similarity matrix A , Ghandehari and Janssen [21] introduced a parameter $\Gamma_1(A)$ and showed that one can construct a Robinson similarity R (with the same order of lines and columns as A) such that $\|A - R\|_1 \leq 26\Gamma_1(A)^{\frac{1}{3}}$. The result of Atkins et al. [3] in the case of the Fiedler vectors with no repeated values was generalized by Fogel et al. [20] to the case when the entries of A are subject to a uniform noise or some entries are not given. Basic examples of Robinson dissimilarities are the ultrametrics. Similarly to the classical bijection between ultrametrics and hierarchies, there is a one-to-one correspondence between Robinson dissimilarities and pseudo-hierarchies due to Diday [15] and Durand and Fichet [16]. Pseudo-hierarchies are classical structures in classification with overlapping classes.

1.2. Paper's organization. The paper is organized as follows. The main notions related to the Robinson property are given in Section 2. In Section 3, we introduce modules and copoints of a dissimilarity space (X, d) and present their basic properties. In particular, we show that all copoints of a given point p define a partition of $X \setminus \{p\}$. In Section 4, we investigate the copoint partitions and the compatible orders for flat and conical Robinson spaces. In Section 5, we investigate the properties of copoint partitions and their (extended) quotients in general Robinson spaces. In Section 6, we introduce the concept of proximity pre-order for an (unknown) compatible order. We show that for extended quotients this pre-order is an order and we show how to retrieve a compatible order from this proximity order. The concepts and the results of Sections 3 and 6 are used in the divide-and-conquer algorithm, described and analyzed in Section 7.

2. Preliminaries. In this section, we give some definitions which will be used throughout this paper. When not defined just before their first use, all notions and notations in this paper are defined here.

2.1. Robinson dissimilarities. Let $X = \{p_1, \dots, p_n\}$ be a set of n elements, called *points*. A *dissimilarity* on X is a symmetric function d from X^2 to the nonnegative real numbers such that $d(x, y) = d(y, x) \geq 0$ and $d(x, y) = 0$ if $x = y$. Then $d(x, y)$ is called the *distance* between x, y and (X, d) is called a *dissimilarity space*. A partial order on X is called *total* if any two elements of X are comparable. Since we will mainly deal with total orders, we abbreviately call them *orders*.

DEFINITION 2.1 (Robinson space). *A dissimilarity d and an order $<$ on X are called compatible if $x < y < z$ implies that $d(x, z) \geq \max\{d(x, y), d(y, z)\}$. If a dissimilarity space (X, d) admits a compatible order, then d is said to be Robinson and (X, d) is called a Robinson space.*

Equivalently, (X, d) is Robinson if its distance matrix $D = (d(p_i, p_j))$ can be symmetrically permuted so that its elements do not decrease when moving away from the main diagonal along any row or column. Such a dissimilarity matrix D is said to have the *Robinson property* [14, 15, 16, 23]. If $Y \subset X$, we denote by $(Y, d|_Y)$ (or simply by (Y, d)) the dissimilarity space obtained by restricting d to Y ; we call (Y, d) a *subspace* of (X, d) . If (X, d) is a Robinson space, then any subspace (Y, d) of (X, d) is also Robinson and the restriction of any compatible order $<$ of X to Y is compatible. If d and $<$ are compatible, then d is also compatible with the order $<^{\text{op}}$ opposite to $<$. Given two dissimilarity spaces (X', d') and (X, d) , a map $\varphi : X' \rightarrow X$ is an *isometric embedding* of (X', d') in (X, d) if for any $x, y \in X'$ we have $d(\varphi(x), \varphi(y)) = d'(x, y)$, i.e., if (X', d') can be viewed as a subspace of (X, d) .

The *ball* of radius $r \geq 0$ centered at $x \in X$ is the set $B_r(x) := \{y \in X : d(x, y) \leq r\}$. The *diameter* of a subset Y of X is $\text{diam}(Y) := \max\{d(x, y) : x, y \in Y\}$ and a pair $x, y \in Y$ such that $d(x, y) = \text{diam}(Y)$ is called a *diametral pair* of Y . A point x of Y is called *non-diametral* if x does not belong to a diametral pair of Y . From the definition of a Robinson dissimilarity follows that d is Robinson if and only if there exists an order $<$ on X such that all balls $B_r(x)$ of (X, d) are intervals of $<$. Moreover, this property holds for all compatible orders. Basic examples of Robinson dissimilarities are the ultrametrics, thoroughly used in phylogeny. Recall, that d is an *ultrametric* if $d(x, y) \leq \max\{d(x, z), d(y, z)\}$ for all $x, y, z \in X$. Another example of a Robinson space is provided by the standard *line-distance* between n points $p_1 < \dots < p_n$ of $\mathbb{R}$. Any line-distance has exactly two compatible orders: the order $p_1 < \dots < p_n$ defined by the coordinates of the points and its opposite.

DEFINITION 2.2 (Flat spaces). *If a Robinson space (X, d) has only two compatible orders $<$ and $<^{\text{op}}$, then (X, d) is said to be flat.*

Line-distances are flat but the converse is not true. We conclude with the definition of conical spaces:

DEFINITION 2.3 (Conical spaces). *A dissimilarity space (X, d) is called conical with apex p if all points of $X \setminus \{p\}$ have the same distance $\delta > 0$ to p , i.e., $d(p, x) = \delta$ for any $x \in X \setminus \{p\}$. Since p has the same distance δ to all points of $X \setminus \{p\}$, (X, d) is a cone over $(X \setminus \{p\}, d)$ with apex p .*

2.2. Algorithms and data structures. Our algorithms will be written in pseudocode. They do not use any fancy data structures besides lists and balanced binary search trees.

We use a bracketed notation for *lists* with $[]$ being the empty list. As a choice of presentation, we will use lists in a persistent (or non-destructive) way [32], meaning that a list cannot be modified once defined. To this end, we introduce the two operators $\cdot$ and $\uplus$ defined by

$$\begin{aligned} x \cdot [l_1, \dots, l_n] &= [x, l_1, \dots, l_n] \\ [l_1, \dots, l_n] \uplus [l'_1, \dots, l'_m] &= [l_1, \dots, l_n, l'_1, \dots, l'_m]. \end{aligned}$$

One can implement the operator $\cdot$ in $O(1)$ time and $\uplus$ in $O(n)$ time (n is the length of the left operand), using single-linked lists. Extracting the first element of a list also takes $O(1)$ time. We will also use the reverse operation with time-complexity $O(n)$, and concatenate with time-complexity $\sum_{i=1}^{k-1} |L_i|$, where:

$$\begin{aligned} \text{reverse}([l_1, l_2, \dots, l_n]) &= [l_n, l_{n-1}, \dots, l_1] \\ \text{concatenate}(L_1, \dots, L_k) &= L_1 \uplus \dots \uplus L_k. \end{aligned}$$

Balanced binary search trees (see e.g. [32]) are used solely to sort in increasing order a list of n elements with at most k distinct key values, in time $O(n \log k)$. This is achieved by building a balanced binary search tree of the key values appearing in the list, each associated to a list of elements sharing that key value. The sorting algorithm consists in inserting each element in the list associated to its key value, then concatenating all the associated lists in increasing order of the key values. Each insertion takes time $O(\log k)$, and the final concatenation takes time $O(n)$. We denote the binary search tree operation by $\text{insert}(T, \text{key}, \text{value})$ (inserts a value with a given comparable key), $\text{containsKey}(T, \text{key})$ (checks whether there is a value with a given key), $\text{get}(T, \text{key})$ (retrieves the value associated to a given key), and $\text{values}(T)$ (returns the list of keys in increasing order of their values).

2.3. Partitions and pre-orders. A *partition* of a set X is a family of sets $\mathcal{P} = \{B_1, \dots, B_m\}$ such that $B_i \cap B_j = \emptyset$ for any $i \neq j$ and $\bigcup_{i=1}^m B_i = X$. The sets $B_1, \dots, B_m$ are called the *classes* of $\mathcal{P}$. A *pre-order*

is a partial order $<$ on X for which incomparability is transitive. A partial order $<$ on X is a pre-order exactly when there exists an ordered partition $\mathcal{R} = (B_1, \dots, B_m)$ of X such that for $x \in B_i$ and $y \in B_j$ we have $x < y$ if and only if $i < j$. Consequently, we will also view a pre-order $<$ as an ordered partition $\mathcal{R} = (B_1, \dots, B_m)$. A partial order $<'$ extends a partial order $<$ if $x < y$ implies $x <' y$ for all $x, y \in X$.

DEFINITION 2.4 (Stable partition). *A partition $\mathcal{P} = \{B_1, \dots, B_m\}$ of a dissimilarity space (X, d) is a stable partition if for any $i \neq j$ and for any three points $x, y \in B_i$ and $z \in B_j$, we have $d(z, x) = d(z, y)$.*

A non-stable partition $\mathcal{P}$ can be transformed into a stable partition by applying the classical operation of *partition refinement*, which proceeds as follows. The algorithm maintains the current partition $\mathcal{P}$ and for each class B of $\mathcal{P}$ maintains the set $Z(B)$ of all points outside B which still have to be processed to refine B . While $\mathcal{P}$ contains a class B with nonempty $Z(B)$, the algorithm pick any point z of $Z(B)$ and partition B into maximal classes that are not distinguishable from z : i.e., for any such new class B' and any $x, x' \in B'$ we have $d(x, z) = d(x', z)$. Finally, the algorithm removes B from $\mathcal{P}$ and insert each new class B' in $\mathcal{P}$ and sets $Z(B') := (B \setminus B') \cup (Z(B) \setminus \{z\})$. Notice that each class B is partitioned into subclasses by comparing the distances of points of B to the point $z \notin B$ and such distances never occur in later comparisons. Also, if the final stable partition has classes $B'_1, \dots, B'_t$, then the distances between points in the same class B'_i are never compared to other distances. This algorithm is formalized in [Algorithms 2.1](#) and [2.2](#), where one would call `partitionRefine( $B, X \setminus B$ )` for each $B \in \mathcal{P}$ to get a stable partition. We will not use [Algorithm 2.1](#), but will introduce and fully analyze a similar algorithm [Algorithm 6.1](#) that returns an ordered partition. It also uses [Algorithm 2.2](#). We now establish the complexity and correctness of [Algorithm 2.2](#).

Algorithm 2.1 `partitionRefine( $B, Z(B)$ )`

Input: a dissimilarity space (X, d) (implicit), a class $B \subseteq X$ and a set $Z(B) \subseteq X \setminus B$

Output: a partition $\{B_1, B_2, \dots, B_k\}$ of B

```

1: if  $Z(B) = \emptyset$  then
2:   return  $\{B\}$ 
3: let  $q \in Z(B)$ ,  $\triangleright$  choose  $q$  to be the first element of  $Z(B)$ 
4: let  $\{B_1, \dots, B_m\} = \text{refine}(q, B)$   $\triangleright$  ignore the order of the  $B_i$ s
5: for  $i \in \{1, \dots, m\}$  do
6:   let  $\mathcal{P}_i = \text{partitionRefine}(B_i, \text{concatenate}(B_1, \dots, B_{i-1}, B_{i+1}, \dots, B_m, Z(B) \setminus \{q\}))$ 
7: return  $\bigcup_{i=1}^m \mathcal{P}_i$ 

```

Algorithm 2.2 `refine( $q, S$ )`

Input: a dissimilarity space (X, d) , a point $q \in X$, a subset $S \subseteq X$.

Output: an ordered partition of S , by increasing distance from q

```

1: let  $T$  be an empty balanced binary tree, with keys in  $\mathbb{N}$ 
2: for  $x \in S$  do
3:   if  $\neg \text{containsKey}(T, d(q, x))$  then
4:     insert( $T, d(q, x), []$ )
5:   insert( $T, d(q, x), x \cdot \text{get}(T, d(q, x))$ )
6: return  $\text{values}(T)$ 

```

LEMMA 2.5. [Algorithm 2.2](#) called on (q, S) outputs a partition $\mathcal{S} = (S_1, \dots, S_m)$ of S in $O(|S| \log m)$ time, where

- (i) for each $1 \leq i \leq m$, for all $x, y \in S_i$, $d(q, x) = d(q, y)$,
- (ii) for each $1 \leq i < j \leq m$, for all $x \in S_i, y \in S_j$, $d(q, x) < d(q, y)$.

Proof. First, $\mathcal{S}$ is a partition, since each element is inserted in a list of T exactly once. Each class of $\mathcal{S}$ is at constant distance from q since we use the distances to q as keys. Finally, the classes of $\mathcal{S}$ are sorted by increasing distances from q , because $\text{values}(T)$ returns its associated values in increasing order of keys. The complexity analysis follows from the fact that the binary search tree contains at most m keys, hence each of its elementary operations are in $O(\log m)$. The evaluation of $\text{values}(T)$ can be done in $O(m)$ operations by

a simple right-to-left DFS traversal of the binary search tree, inserting (not appending) each list in T from farthest to closest into the returned list. $\square$

2.4. The running example. Throughout the paper, we will use the dissimilarity space in [Figure 2.1](#) and some of its subspaces to illustrate the algorithms and the introduced notions. As will be seen in the final [Figure 7.1](#), this dissimilarity space is Robinson, with the compatible order (among others): $19 < 5 < 15 < 2 < 12 < 13 < 14 < 11 < 4 < 3 < 18 < 8 < 16 < 9 < 1 < 17 < 10 < 6 < 7$.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	0	10	9	9	10	8	9	9	4	8	9	10	9	9	10	9	4	9	10
2		0	8	8	2	10	11	8	10	10	5	2	5	5	1	8	10	8	2
3			0	1	8	9	9	2	9	9	6	8	6	6	8	2	9	2	8
4				0	8	9	9	2	9	9	6	8	6	6	8	3	9	2	8
5					0	10	11	8	10	10	5	3	5	5	2	8	10	8	1
6						0	9	9	9	7	9	10	9	9	10	9	8	9	10
7							0	9	9	9	9	11	9	9	11	9	9	9	11
8								0	9	9	6	8	6	6	8	2	9	2	8
9									0	8	9	10	9	9	10	9	6	9	10
10										0	9	10	9	9	10	9	7	9	10
11											0	5	1	1	5	6	9	6	5
12												0	5	5	2	8	10	8	4
13													0	1	5	6	9	6	5
14														0	5	6	9	6	5
15															0	8	10	8	2
16																0	9	2	8
17																	0	9	10
18																		0	8
19																			0

FIG. 2.1. A distance matrix D of a Robinson space (X, d) with $X = \{1, \dots, 19\}$.

To illustrate the notions of flat and conical subspaces, notice that the subspace $\{5, 14, 3, 9, 7\}$ is flat, with compatible orders $5 < 14 < 3 < 9 < 7$ and its reverse. This follows from the fact that $\{5, 7\}$ is the unique diametral pair, whence 5 and 7 must be the extremities of any compatible order. Then sorting the remaining points by their distances from 5 imposes the rest of the order. One can check in [Figure 2.2](#). Notice also that the subspace $\{1, 6, 7, 9, 10\}$ is conical with apex 7 and $\delta = 9$.

The stable partition algorithm applied to the partition $\{\{2, 5\}, \{1, 3, 4, 6, 7, 8\}\}$ will return the partition $\{\{2, 5\}, \{3, 4, 8\}, \{1, 6, 7\}\}$. This is done by using 2 as a pivot on $\{1, 3, 4, 5, 7, 8\}$, because $d(2, \{3, 4, 8\}) = 8$ while $d(2, \{1, 6\}) = 10$ and $d(2, \{7\}) = 11$. One can check that this partition is stable, see [Figure 2.2](#).

														2	5	3	4	8	1	6	7	
	5	14	3	9	7			1	6	7	9	10		2	0	2	8	8	8	10	10	11
5	0	5	8	10	11		1	0	8	9	4	8		5		0	8	8	8	10	10	11
14		0	6	9	9		6		0	9	9	7					0	1	2	9	9	9
3			0	9	9		7			0	9	9						0	2	9	9	9
9				0	9		9				0	8							0	9	9	9
7					0		10					0								0	8	9
														1						0	8	9
														6							0	9
														7								0

FIG. 2.2. An illustration of several subspaces of (X, d) , from left to right: a flat subspace, a conical subspace with apex 7, a subspace with an explicit stable partition.

3. Mmodules in dissimilarity spaces. In this section, we introduce and investigate the notion of mmodule. As one can see from their use in this paper, our motivation for introducing them stems from the property of classes in stable partitions: the points of the same class C cannot be distinguished from the outside, i.e., for any two points $x, y \in C$ and any point $z \notin C$, the equality $d(z, x) = d(z, y)$ holds. After having obtained the main properties of mmodules in general dissimilarities presented in [Subsection 3.1](#), we discovered that our mmodules coincide with “clans” in symmetric 2-structures, defined and investigated by Ehrenfeucht and Rozenberg [18, 19] (see also Chein, Habib and Maurer [10]). Since their theory is developed in a more general non-symmetric setting, we prefer to give a self-contained presentation of elementary properties of mmodules. Applying an argument from abstract convexity, we deduce that for each point p

all maximal by inclusion mmodules not containing p together with p define a partition of X . This copoint partition is used in our divide-and-conquer algorithm for recognizing Robinson spaces.

3.1. Mmodules. We continue with the definition of mmodule of a dissimilarity space (X, d) .

DEFINITION 3.1 (Mmodules). *A set $M \subseteq X$ is called an mmodule (a metric module or a matrix module, pronounced [ɛm 'mɒdju:l]) if for any $z \in X \setminus M$ and all $x, y \in M$ we have $d(z, x) = d(z, y)$. Denote by $\mathcal{M} = \mathcal{M}(X, d)$ the set of all mmodules of (X, d) . An mmodule M is called maximal if M is a maximal by inclusion mmodule different from X .*

In graph theory, the subgraphs indistinguishable from the outside are called modules (see [17, 22]), explaining our choice of the term “mmodule”. Trivially, $\emptyset, X$, and $\{p\}, p \in X$ are mmodules; we call them *trivial mmodules*.

The running example. The sets $\{1, 6, 9, 10, 17\}, \{2, 5, 12, 15, 19\}, \{3, 4, 8, 16, 18\}, \{7\}, \{13, 14, 15\}$ are the maximal mmodules of the running example. The set $\{12, 5, 19\}$ is a non-maximal mmodule.

We continue with the basic properties of mmodules.

PROPOSITION 3.2. *The set $\mathcal{M} = \mathcal{M}(X, d)$ has the following properties:*

- (i) $M_1, M_2 \in \mathcal{M}$ implies that $M_1 \cap M_2 \in \mathcal{M}$;
- (ii) if $M \in \mathcal{M}$ and $M' \subset M$, then $M' \in \mathcal{M}$ if and only if M' is an mmodule of (M, d) ;
- (iii) if $M_1, M_2 \in \mathcal{M}$ and $M_1 \cap M_2 \neq \emptyset$, then $M_1 \cup M_2 \in \mathcal{M}$, furthermore, if $M_1 \setminus M_2 \neq \emptyset$ and $M_2 \setminus M_1 \neq \emptyset$, then $M_1 \setminus M_2, M_2 \setminus M_1, M_1 \triangle M_2 \in \mathcal{M}$;
- (iv) the union $M_1 \cup M_2$ of two intersecting maximal mmodules $M_1, M_2 \in \mathcal{M}$ is X ;
- (v) if M_1 and M_2 are two disjoint maximal mmodules and M is a nontrivial mmodule contained in $M_1 \cup M_2$, then either $M \subset M_1$ or $M \subset M_2$;
- (vi) if $M_1, M_2 \in \mathcal{M}$ and $M_1 \cap M_2 = \emptyset$, then $d(u, v) = d(u', v')$ for any (not necessarily distinct) points $u, u' \in M_1$ and $v, v' \in M_2$;
- (vii) if $\mathcal{M}'$ is any partition of X into mmodules, then $\mathcal{M}'$ is a stable partition.

Proof. To (i): Pick any $x \notin M_1 \cap M_2$ and $u, v \in M_1 \cap M_2$. If $x \notin M_1 \cup M_2$, then $d(x, u) = d(x, v)$ since $M_1, M_2 \in \mathcal{M}$. If $x \in M_1 \cup M_2$, say $x \in M_2 \setminus M_1$, then $d(x, u) = d(x, v)$ since $M_1 \in \mathcal{M}$.

To (ii): First, let M' be an mmodule of (X, d) . This implies that $d(x, u) = d(x, v)$ for any $x \in M \setminus M'$ and $u, v \in M'$, thus M' is an mmodule of (M, d) . Conversely, let M' be an mmodule of (M, d) and we assert that M' is an mmodule of (X, d) . Pick any $x \in X \setminus M'$ and $u, v \in M'$. If $x \in X \setminus M$, then $d(x, u) = d(x, v)$ since $u, v \in M' \subset M$ and M is an mmodule. If $x \in M \setminus M'$, then $d(x, u) = d(x, v)$ since M' is an mmodule of (M, d) and we are done.

To (iii): We first show that $M_1 \cup M_2 \in \mathcal{M}$. If $M_1 \cup M_2 = X$, we are done. Otherwise, pick any $x \in X \setminus (M_1 \cup M_2)$ and $u, v \in M_1 \cup M_2$. If $u, v \in M_1$ or $u, v \in M_2$, then $d(x, u) = d(x, v)$ because $M_1, M_2 \in \mathcal{M}$. Thus, let $u \in M_1 \setminus M_2$ and $v \in M_2 \setminus M_1$. Pick any $w \in M_1 \cap M_2$. Then $d(x, u) = d(x, w)$ and $d(x, v) = d(x, w)$ since M_1 and M_2 are mmodules. Consequently, $d(x, u) = d(x, v)$ and thus $M_1 \cup M_2 \in \mathcal{M}$.

Since $M_1, M_2 \in \mathcal{M}$, for any $u, v \in M_1 \setminus M_2, u', v' \in M_2 \setminus M_1, y \in M_1 \cap M_2$, and $x \in X \setminus (M_1 \cup M_2)$, we have $d(x, u) = d(x, v) = d(x, y) = d(x, u') = d(x, v')$ and $d(u, u') = d(v, v') = d(u, y) = d(v, y) = d(u', y) = d(v', y)$. This shows that $M_1 \setminus M_2, M_2 \setminus M_1$, and $M_1 \triangle M_2$ are mmodules.

To (iv): This is a direct consequence of (iii) and the definition of maximal mmodules.

To (v): Since M is nontrivial, if M is not equal to one of the M_i 's ($i \in \{1, 2\}$), then we have $\emptyset \neq M \cap M_i \subsetneq M_i$ for, say, $i = 2$. If $M \not\subset M_2$, then $M \cap M_1 \neq \emptyset$ and thus, by (iii), $M \cup M_1$ is an mmodule which, as $M_1 \subsetneq M \cup M_1 \neq X$, contradicts the maximality of M_1 .

To (vi): Since M_2 is an mmodule and $u \notin M_2, d(u, v) = d(u, v')$. Since M_1 is an mmodule and $v' \notin M_1, d(v', u) = d(v', u')$. Consequently, $d(u, v) = d(u', v')$.

To (vii): This follows from the definition of mmodules and the fact that $\mathcal{M}'$ partitions X . $\square$

By **Proposition 3.2(i)**, $\mathcal{M}$ is closed by intersection, thus $(X, \mathcal{M})$ is a *convexity structure* [38]. Thus for each subset A of X we can define the *convex hull* $\text{mconv}(A)$ of A as the smallest mmodule containing A : $\text{mconv}(A)$ is the intersection of all mmodules containing A . For points $u, v \in X$, we call $\langle u, v \rangle := \{x \in X : d(x, u) \neq d(x, v)\}$ the *interval* between u and v .

LEMMA 3.3. $\langle u, v \rangle \subseteq \text{mconv}(u, v)$.

Proof. Pick x in $\langle u, v \rangle$. If $x \notin \text{mconv}(u, v)$, then $d(x, u') = d(x, v')$ for any $u', v' \in \text{mconv}(u, v)$. This is impossible since $u, v \in \text{mconv}(u, v)$ and $d(x, u) \neq d(x, v)$ by the definition of $\langle u, v \rangle$. $\square$

The converse inclusion is not true. However, the following lemma shows that $\mathcal{M}$ is an *interval convexity* [38] in the following sense:

LEMMA 3.4. $A \subseteq X$ is an mmodule if and only if $\langle u, v \rangle \subseteq A$ for any two points $u, v \in A$.

Proof. By Lemma 3.3, $\langle u, v \rangle \subseteq \text{mconv}(u, v)$. Since mconv is a convexity operator, $\text{mconv}(u, v) \subseteq \text{mconv}(A)$. Thus, if $A \in \mathcal{M}$, then $\langle u, v \rangle \subseteq \text{mconv}(A) = A$. Conversely, suppose $\langle u, v \rangle \subseteq A$ for any $u, v \in A$. If A is not an mmodule, there exist $x \in S \setminus A$ and $u, v \in A$ such that $d(x, u) \neq d(x, v)$. But this implies that x belongs to $\langle u, v \rangle \subseteq A$, contrary to the choice of x . $\square$

3.2. Copoint partition. We continue by defining copoints. This term arises from abstract convexity [27, 38]. Then we prove that the copoints attached to any point p of (X, d) are pairwise disjoint.

DEFINITION 3.5 (Copoint). A copoint at a point p (or a p -copoint) is any mmodule C of (X, d) that is maximal by inclusion among the mmodules not containing p ; the point p is the attaching point of C .

The running example. The copoints of point 1 are $C_1 = \{9\}$, $C_2 = \{17\}$, $C_3 = \{6\}$, $C_4 = \{10\}$, $C_5 = \{3, 4, 8, 16, 18\}$, $C_6 = \{7\}$, $C_7 = \{11, 13, 14\}$ and $C_8 = \{2, 5, 12, 15, 19\}$. This can be readily checked using Figure 3.2.

The copoints of $\mathcal{M}$ minimally generate $\mathcal{M}$, in the sense that each mmodule M is the intersection of the copoints containing M [38]. Denote by $\mathcal{C}_p$ the set of all copoints at p plus the trivial mmodule $\{p\}$.

LEMMA 3.6. For any $p \in X$, $\mathcal{C}_p$ defines a partition of X .

Proof. Pick any copoints C, C' at p . If $C \cap C' \neq \emptyset$, by Proposition 3.2(iii), the union $C \cup C'$ is an mmodule not containing p , contrary to the assumption that C, C' are copoints at p . Since for any point $q \neq p$, $\{q\}$ is an mmodule, q is contained in a copoint at p . Thus $\mathcal{C}_p$ defines a partition of X . $\square$

DEFINITION 3.7 (Copoint partition). Consequently, we call $\mathcal{C}_p := \{C_0 := \{p\}, C_1, \dots, C_k\}$ the copoint partition of (X, d) with attaching point p .

From Proposition 3.2(vii) it follows that $\mathcal{C}_p$ is a stable partition of X (see Definition 2.4). $\mathcal{C}_p$ can be constructed by applying the stable partition algorithm to the initial partition $\{\{p\}, X \setminus \{p\}\}$.

DEFINITION 3.8 (Trivial and cotrivial copoint partitions). The copoint partition $\mathcal{C}_p$ is called trivial if $\mathcal{C}_p$ consists only of the points of X , i.e., $\mathcal{C}_p = \{\{x\} : x \in X\}$, and cotrivial if $\mathcal{C}_p = \{\{p\}, X \setminus \{p\}\}$, i.e., all points of $X \setminus \{p\}$ have the same distance to p . If $\mathcal{C}_p$ is trivial, then (X, d) is called p -trivial.

Notice that the copoint partition $\mathcal{C}_p$ is cotrivial if and only if (X, d) is conical with apex p (see Definition 2.3). The following result follows directly from the definitions:

LEMMA 3.9. For a dissimilarity space (X, d) , the following holds:

- (i) (X, d) is p -trivial for all $p \in X$ if and only if all mmodules of (X, d) are trivial;
- (ii) (X, d) is conical for all $p \in X$ if and only if $d(x, y) = \delta$ for all $x \neq y$ and some $\delta > 0$;
- (iii) if (X, d) is conical with apex p , then each mmodule of $(X \setminus \{p\}, d)$ is an mmodule of (X, d) .

The heart of our divide-and-conquer algorithm is a decomposition of the dissimilarity (X, d) into the dissimilarities of its copoints, on which we recurse. The merge step on the other hand will use the quotient space:

DEFINITION 3.10 (Quotient space). Let $\mathcal{C}_p = \{C_0 = \{p\}, C_1, \dots, C_k\}$. The quotient space $(\mathcal{C}_p, \hat{d})$ of (X, d) has the classes of $\mathcal{C}_p$ as points and for $C_i, C_j, i \neq j$ of $\mathcal{C}_p$ we set $\hat{d}(C_i, C_j) := d(u, v)$ for an arbitrary pair $u \in C_i, v \in C_j$.

The running example. Considering also $C_0 = \{1\}$, we get a quotient space $(\mathcal{C}_1, \hat{d})$ given in Figure 3.1.

LEMMA 3.11. The quotient space $(\mathcal{C}_p, \hat{d})$ is C_0 -trivial.

Proof. Let $\mathcal{C}_p = \{C_0 = \{p\}, C_1, \dots, C_k\}$ and suppose that $(\mathcal{C}_p, \hat{d})$ has a non-trivial mmodule M not containing C_0 . For any $C_j, C_{j'} \in M$ and $C_i \in \mathcal{C}_p \setminus M$, we have $\hat{d}(C_i, C_j) = \hat{d}(C_i, C_{j'})$. Setting $Y := \bigcup_M C_i$,

	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
C_0	0	4	4	8	8	9	9	9	10
C_1		0	6	9	8	9	9	9	10
C_2			0	8	7	9	9	9	10
C_3				0	7	9	9	9	10
C_4					0	9	9	9	10
C_5						0	9	6	8
C_6							0	9	11
C_7								0	5
C_8									0

FIG. 3.1. The quotient space $(C_1, \hat{d})$

for any $x, y \in Y$ and $z \in X \setminus Y$, we have $d(z, x) = d(z, y)$. Consequently, Y is an mmodule of (X, d) not containing p , contradicting the maximality of the C_i 's. $\square$

The definition of the quotient space implies that one can permute the rows and columns of D to get the following nice property. Partition the rows and the columns of the distance matrix D of (X, d) into sets corresponding to the copoints of $\mathcal{C}_p$, and permute the rows and the columns of D , starting with the rows and columns corresponding to the first copoint $C_0 = \{p\}$ of $\mathcal{M}$, then to the second copoint C_1 of $\mathcal{C}_p$, etc. Then, in the resulting permuted matrix D' , for each pair $C_i, C_j, i \neq j$, of $\mathcal{C}_p$, the entries of D' corresponding to rows from C_i and columns from C_j and rows from C_j and columns from C_i are all equal to $\hat{d}(C_i, C_j)$. This provides a block decomposition of D' such that in each rectangle not intersecting the main diagonal of D' all entries are equal. The rectangles intersecting the main diagonal are squares defined by the entries located at the intersections of rows and columns corresponding to a copoint C_i . Therefore the recursive call to $C_i \in \mathcal{C}_p$ corresponds to dealing with the dissimilarity space (C_i, d) defined by the entries in this diagonal square. The dissimilarity matrix of $(\mathcal{C}_p, \hat{d})$ is obtained from D' by replacing each $|C_i| \times |C_j|$ rectangle by a single entry $\hat{d}(C_i, C_j)$ and contracting each copoint of $\mathcal{C}_p$ to a single point. This illustrates how the dissimilarity space is decomposed into the dissimilarities of each copoint (the diagonal blocks) plus the quotient space (representing the non-diagonal blocks).

The running example. The copoints of point 1 give out the block decomposition of [Figure 3.2](#).

	1	9	17	6	10	3	4	8	16	18	7	11	13	14	2	5	12	15	19
1	0	4	4	8	8	9	9	9	9	9	9	9	9	9	10	10	10	10	10
9		0	6	9	8	9	9	9	9	9	9	9	9	9	10	10	10	10	10
17			0	8	7	9	9	9	9	9	9	9	9	9	10	10	10	10	10
6				0	7	9	9	9	9	9	9	9	9	9	10	10	10	10	10
10					0	9	9	9	9	9	9	9	9	9	10	10	10	10	10
3						0	1	2	2	2	9	6	6	6	8	8	8	8	8
4							0	2	3	2	9	6	6	6	8	8	8	8	8
8								0	2	2	9	6	6	6	8	8	8	8	8
16									0	2	9	6	6	6	8	8	8	8	8
18										0	9	6	6	6	8	8	8	8	8
7											0	9	9	9	11	11	11	11	11
11												0	1	1	5	5	5	5	5
13													0	1	5	5	5	5	5
14														0	5	5	5	5	5
2															0	2	2	1	2
5																0	3	2	1
12																	0	2	4
15																		0	2
19																			0

FIG. 3.2. A permuted matrix for the copoints attached at 1.

3.3. Tree representations of mmodules. A family of subsets $\{M_1, \dots, M_k\}$ of X is a *copartition* of X if $\{X \setminus M_1, \dots, X \setminus M_k\}$ is a partition of X . For a set $M \subseteq X$, let $\overline{M} := X \setminus M$. We denote by $\widehat{\mathcal{M}} := \widehat{\mathcal{M}}(X, d)$ the set of all maximal mmodules of (X, d) .

LEMMA 3.12. $\widehat{\mathcal{M}}$ is a partition or a copartition of X .

Proof. If the maximal mmodules are pairwise disjoint, then $\widehat{\mathcal{M}}$ is a partition. We assume now that there exist intersecting maximal mmodules M and M' . Then $M \cup M' = X$ by Proposition 3.2(iv). We assert that every pair of maximal mmodules intersects. Let $M_1, M_2 \in \widehat{\mathcal{M}}$ and suppose M_1 and M_2 are disjoint. Since $M \cup M' = X$, we may assume $M \cap M_1 \neq \emptyset$. By Proposition 3.2(iv), $M \cup M_1 = X$, and then $M_2 \subseteq M$. By maximality $M_2 = M$, contradicting the fact that M_1 and M_2 are disjoint. Hence any two maximal mmodules M_1 and M_2 intersect, yielding $M_1 \cup M_2 = X$ and $\overline{M_1} \cap \overline{M_2} = \emptyset$.

Let $A = \bigcap \widehat{\mathcal{M}}$ be the intersection of all maximal mmodules, and suppose that A is not empty. Then, as $X = \bigcup \widehat{\mathcal{M}}$, we can write $\overline{A} = \bigcup_{M_1, M_2 \in \widehat{\mathcal{M}}} M_1 \setminus M_2$, which by Proposition 3.2(iii) implies that $\overline{A}$ is an mmodule. By definition of A , A is contained in every maximal mmodule, and by assumption $\overline{A} \neq X$, thus $A \cup \overline{A} = X$ is contained in a maximal mmodule, contradiction. Thus A is empty. This proves that $\overline{A} = \bigcup \{\overline{M} : M \in \widehat{\mathcal{M}}\} = X$, and we know that all these sets are disjoint, so $\widehat{\mathcal{M}}$ is a copartition of X . $\square$

Lemma 3.12 describes the structure of maximal mmodules. To extend that structure to all mmodules, we must understand how non-maximal modules relates to maximal mmodules. In the case of a partition, this is settled by Proposition 3.2(v). The case of copartitions is the goal of the next Lemma.

LEMMA 3.13. If $\widehat{\mathcal{M}} = \{M_1, \dots, M_k\}$ is a copartition, then for any mmodule $M \in \mathcal{M}$, either there is $J \subseteq \{1, 2, \dots, k\}$ such that $M = \bigcup_{j \in J} \overline{M_j}$ or there is $i \in \{1, 2, \dots, k\}$ such that $M \subset \overline{M_i}$.

Proof. Let M be an mmodule and suppose that M intersects the complements of two maximal mmodules, say $M \cap \overline{M_1} \neq \emptyset$ and $M \cap \overline{M_2} \neq \emptyset$. Since $M_1 \cap M \neq \emptyset$, by Proposition 3.2(iii), $M_1 \cup M$ is an mmodule which strictly contains M_1 . By maximality of M_1 , $M_1 \cup M = X$ and $\overline{M_1} \subseteq M$. Consequently, $M = \bigcup \{\overline{M_i} : M \cap \overline{M_i} \neq \emptyset\}$, proving the assertion. $\square$

DEFINITION 3.14. Given a set X , a $\cup\cap$ -tree on X is a tree $\mathcal{T}$ with leaf set X and inner nodes labelled by $\cup$ or $\cap$ which represents a subset $\mathcal{S}(\mathcal{T})$ of the power set $\mathcal{P}(X)$:

- (i) the set of leaves of any node of $\mathcal{T}$ is in $\mathcal{S}(\mathcal{T})$,
- (ii) if a node N is labelled $\cap$, then the set of leaves of the union of any subset of children of N is in $\mathcal{S}(\mathcal{T})$.

The $\cup$ -nodes get their name from the fact that they represent partitions $M_1 \cup M_2 \cup \dots \cup M_k$, where each child corresponds to a part, while the $\cap$ -nodes represent copartitions, where each child is the intersection of all maximal mmodules except one.

PROPOSITION 3.15. Let (X, d) be a dissimilarity space. There exists a unique $\cup\cap$ -tree $\mathcal{T}_{\mathcal{M}}$ on X (up to reordering the children of each node) such that $\mathcal{M}(X, d) = \mathcal{S}(\mathcal{T}_{\mathcal{M}})$. $\mathcal{T}_{\mathcal{M}}$ is called the mmodule-tree of (X, d) .

Proof. If $\widehat{\mathcal{M}}$ is a partition, then the root of $\mathcal{T}_{\mathcal{M}}$ has label $\cup$ and its children are the trees defined inductively for each maximal mmodule. If $\widehat{\mathcal{M}}$ is a copartition, then the root of $\mathcal{T}_{\mathcal{M}}$ has label $\cap$ and its children are the trees defined inductively for complements of maximal mmodules. By Lemma 3.12, this procedure defines a tree, whence it only remains to establish (i) and (ii). These properties hold for maximal mmodules. Pick now a non-maximal mmodule M . By Proposition 3.2(v), if $\widehat{\mathcal{M}}$ is a partition, M is contained in a maximal mmodule M' associated with some child of the root. By induction hypothesis, M is represented in that child. If $\widehat{\mathcal{M}}$ is a copartition, by Lemma 3.13, either M is the union of the complements of maximal mmodules, which corresponds to (ii), or M is strictly contained in the complement of some maximal mmodule M'' , and $\overline{M''}$ is represented as a child of the root. By induction hypothesis, M is represented in that child. $\square$

The running example. The mmodule-tree for the running example is given in Figure 3.3. Since the root is a $\cup$ -node, the maximal mmodules are the leafsets of its children. The maximal mmodules are the leafsets of its children, i.e. the sets $\{19, 12, 5, 15, 2\}$, $\{14, 13, 11\}$, $\{18, 8, 16, 4, 3\}$, $\{7\}$ and $\{10, 9, 17, 6, 1\}$. Conversely, the set $\{7, 10, 9, 17, 6, 1\}$ is not an mmodule. Since α is a $\cap$ -node, the sets $\{18\}$, $\{8\}$, $\{16, 4, 3\}$, $\{18, 8\}$, $\{18, 16, 4, 3\}$ and $\{8, 16, 4, 3\}$ are mmodules. Notice that, since β has two children, we can consider it as either a $\cap$ -node or a $\cup$ -node.

===== maximal mmodules are the leafsets of its children, *i.e.* the sets $\{19, 12, 5, 15, 2\}$, $\{14, 13, 11\}$, $\{18, 8, 16, 4, 3\}$, $\{7\}$ and $\{10, 9, 17, 6, 1\}$. Conversely, the set $\{7, 10, 9, 17, 6, 1\}$ is not an mmodule. Since α is a $\cap$ -node, the sets $\{18\}$, $\{8\}$, $\{16, 4, 3\}$, $\{18, 8\}$, $\{18, 16, 4, 3\}$ and $\{8, 16, 4, 3\}$ are mmodules. Notice that, since β has two children, we can consider it as either a $\cap$ -node or a $\cup$ -node.

~~~~~.r1111

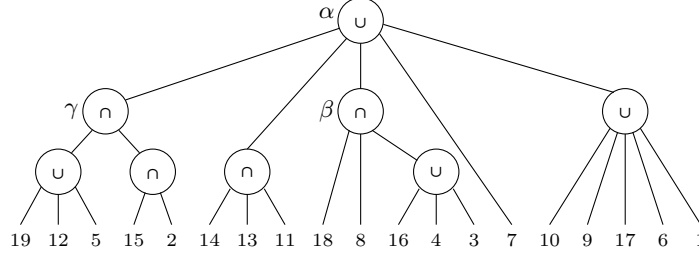


FIG. 3.3. The mmodule-tree  $\mathcal{T}_M$  of the running example. Since  $\alpha$  is a  $\cup$ -node, the sets  $\{19, 12, 5, 15, 2\}$ ,  $\{14, 13, 11\}$ ,  $\{18, 8, 16, 4, 3\}$ ,  $\{7\}$  and  $\{10, 9, 17, 6, 1\}$  are mmodules, but not the set  $\{7, 10, 9, 17, 6, 1\}$ . Since  $\beta$  is a  $\cap$ -node, the sets  $\{18\}$ ,  $\{8\}$ ,  $\{16, 4, 3\}$ ,  $\{18, 8\}$ ,  $\{18, 16, 4, 3\}$  and  $\{8, 16, 4, 3\}$  are mmodules. Notice that, since  $\gamma$  has two children, we can consider it as either a  $\cap$ -node or a  $\cup$ -node.

**4. Flat and conical Robinson spaces.** In this section, we first study the copoint partitions in flat Robinson spaces. They loosely correspond to the Robinson spaces for which our algorithm find a compatible order without recursion. For conical Robinson spaces we show how to derive compatible orders from a compatible order of its subspace obtained by removing the apex. The importance of conical Robinson spaces stems from the observation that each copoint of  $\mathcal{C}_p$  together with  $p$  define a conical subspace with apex  $p$ .

**4.1. Copoint partitions in flat Robinson spaces.** Copoints in flat Robinson spaces (see [Definition 2.2](#)) are characterized by the following result:

**PROPOSITION 4.1.** *If  $(X, d)$  is a flat Robinson space, then either all copoint partitions of  $(X, d)$  are trivial or there exists a (unique) non-diametral point  $p$  of  $X$  such that  $(X, d)$  is conical with apex  $p$  and all mmodules of  $(X \setminus \{p\}, d)$  are trivial.*

*Proof.* Let  $n = |X| > 2$ . We order  $X$  by a compatible order  $q_1 < q_2 < \dots < q_n$ . Let  $M$  be an mmodule of  $(X, d)$ . Let  $i = \min\{k \in \{1, \dots, n\} : q_k \in M\}$  and  $j = \max\{k \in \{1, \dots, n\} : q_k \in M\}$ . Consider the order  $<'$  obtained from  $<$  by reversing the order between the elements in  $\{q_i, \dots, q_j\}$ :

$$q_1 <' q_2 <' \dots <' q_{i-1} <' q_j <' q_{j-1} <' \dots <' q_i <' q_{j+1} <' q_{j+2} <' \dots <' q_n.$$

We assert that  $<'$  is a compatible order. Indeed, let  $q_x <' q_y <' q_z$ , and assume that they are not in the same order as in  $<$  or  $<^{\text{op}}$ . Hence  $x < i \leq z < y \leq j$  (or symmetrically  $i \leq y < x \leq j < z$ ). Then  $q_x < q_i \leq q_z < q_y \leq q_j$ , from which we get  $d(q_x, q_i) \leq d(q_x, q_z) \leq d(q_x, q_y) \leq d(q_x, q_j) = d(q_x, q_i)$ , and then  $d(q_y, q_z) \leq d(q_x, q_z) = d(q_x, q_y)$ , proving the compatibility of  $<'$ .

Since  $(X, d)$  is flat,  $<$  and  $<'$  are either equal or reverse to each other. In the first case, this means that  $i = j$  and  $M$  is trivial. In the second case, this means that  $i = 1$  and  $j = n$ . So every non trivial mmodule of  $(X, d)$  contains  $q_1$  and  $q_n$ . Suppose now that there are  $\alpha < \beta$  in  $\{2, \dots, n-1\}$  with  $q_\alpha, q_\beta \notin M$ . Then  $d(q_1, q_\alpha) \leq d(q_1, q_\beta) = d(q_\beta, q_n) \leq d(q_\alpha, q_n) = d(q_1, q_\alpha)$ , implying that those quantities are all equal to the same value  $\delta$ . From this,  $d(u, v) = \delta$  for each  $u \in M$ ,  $v \notin M$ , hence  $X \setminus M$  is also an mmodule. As it does not contain  $q_1$  and  $q_n$ ,  $X \setminus M$  is a trivial mmodule, hence  $|M| = n - 1$ .

Consequently, any non-trivial mmodule of  $(X, d)$  is of the form  $X \setminus \{q_i\}$  for some  $i \in \{2, \dots, n-1\}$ . Suppose that  $(X, d)$  admits two non-trivial mmodules  $X \setminus \{q_i\}$  and  $X \setminus \{q_j\}$  with  $1 < i < j < n$  (notice that we need  $n \geq 4$ ). Then for all  $x \in X \setminus \{q_i, q_j\}$  we have  $d(x, q_i) = d(q_i, q_j) = d(q_j, x)$ , hence  $\{q_i, q_j\}$  is an mmodule. Since  $n > 3$ , this is a contradiction to the fact that the non-trivial mmodules have cardinality  $n - 1$ . This proves that  $M$  is unique.

Finally, let  $\Delta = \text{diam}(X) = d(q_1, q_n)$  be the diameter of  $(X, d)$ , let  $j$  be such that  $M = X \setminus \{q_j\}$ . Suppose that  $q_j$  is the end of a diametral pair, that is  $d(q_i, q_j) = \Delta$  for any  $q_i \in M$ . Then for all  $i \in \{1, \dots, j-1\}$

and all  $k \in \{j+1, \dots, n\}$ , we have  $\Delta = d(q_i, q_j) \leq d(q_i, q_k) \leq d(q_1, q_n) = \Delta$ , implying that  $\{q_1, \dots, q_j\}$  is a non-trivial module not containing  $q_n$ , a contradiction.  $\square$

**4.2. Conical Robinson spaces.** For a conical Robinson space  $(X, d)$  with apex  $p$  (see [Definition 2.3](#)), let  $d(p, x) = \delta$  for any  $x \in X \setminus \{p\}$  and  $X' = X \setminus \{p\}$ . We will show how to compute, from any compatible order  $<'$  of  $(X', d)$ , a compatible order  $<$  of  $(X, d)$ .

Let  $<'$  be a compatible order of  $(X', d)$ . Let  $x_*$  and  $x^*$  be respectively the minimal and maximal points of  $<'$ . By a *hole* of  $<'$  we will mean any pair  $(y, z)$  of consecutive points  $y, z \in X'$  of  $<'$  with  $y <' z$ . Informally speaking, a hole is a place where one can insert the point  $p$  and still get a total order. We will also consider the pair  $(x^*, x_*)$  as a hole (this corresponds to inserting  $p$  before or after  $X'$ ). For a hole  $(y, z) \neq (x^*, x_*)$ , let  $<_{(y,z)}$  be the total order obtained by inserting  $p$  in the hole  $(y, z)$ , i.e., by setting  $u <_{(y,z)} v$  when  $u <' v$  if  $u, v \in X'$ , and  $u <_{(y,z)} p$ ,  $p <_{(y,z)} v$  for any  $u, v \in X'$  such that  $u \leq' y$  and  $z \leq' v$ . If  $(y, z) = (x^*, x_*)$ , then we set  $v <_{(y,z)} p$  for all  $v \in X'$ , which means that  $p$  is located after  $x^*$ . We could have equally set  $p <_{(y,z)} u$  for all  $u \in X'$  which would have meant that  $p$  is located before  $x_*$ . This choice is arbitrary. We will call a hole  $(y, z)$  of  $<'$  *admissible* if  $<_{(y,z)}$  is a compatible order of  $(X, d)$ .

**LEMMA 4.2.** *Let  $<'$  be a compatible order of  $(X', d)$ . A hole  $(y, z) \neq (x^*, x_*)$  of  $<'$  is admissible if and only if for any  $u, v \in X'$  with  $u <' v$ , the following conditions hold:*

- (1)  $d(u, v) \geq \delta$  if  $u \leq' y$  and  $z \leq' v$ ;
- (2)  $d(u, v) \leq \delta$  if  $v \leq' y$  or  $z \leq' u$ .

*Proof.* Consider a hole  $(y, z) \neq (x^*, x_*)$ . Pick any three points  $u, v, w \in X$  such that  $u <_{(y,z)} v <_{(y,z)} w$ . If  $p \notin \{u, v, w\}$ , then  $u <' v <' w$  and thus  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$ . Now, let  $p \in \{u, v, w\}$ . First suppose that  $p = u$  (the case  $p = w$  is similar). Then  $d(u, v) = d(u, w) = \delta$ . Consequently,  $d(v, w) \leq d(u, w)$  if and only if  $d(v, w) \leq \delta$ , i.e., condition (2) holds. Now suppose that  $p = v$ . Then  $d(u, v) = d(v, w) = \delta$ . Then  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$  if and only if  $d(u, w) \geq \delta$ , i.e., condition (1) holds. Consequently, the hole  $(y, z)$  is admissible if and only if both conditions (1) and (2) hold.  $\square$

**LEMMA 4.3.** *Let  $<'$  be a compatible order of  $(X', d)$ . The hole  $(x^*, x_*)$  is admissible if and only if  $d(x_*, x^*) \leq \delta$ . Moreover in that case, a hole  $(y, z) \neq (x^*, x_*)$  is admissible if and only if  $d(y, z) = \delta$ .*

*Proof.* Since  $<'$  is a compatible order of  $(X', d)$  and for any  $u, v \in X'$  we have  $x_* \leq' u <' v \leq' x^*$  or  $x_* \leq' v <' u \leq' x^*$ , we conclude that  $d(x_*, x^*) \geq d(u, v)$ . Therefore  $(x^*, x_*)$  is admissible if and only if  $\delta \geq d(x_*, x^*)$ . The second part of the lemma is a consequence of [Lemma 4.2](#), as condition (2) is implied by the fact that the diameter is  $\delta$ .  $\square$

[Lemma 4.3](#) characterizes admissible holes when  $d(x_*, x^*) \leq \delta$ . In this case, an admissible hole can be computed in  $O(1)$ . The next result provides such a characterization when  $d(x_*, x^*) > \delta$ .

**LEMMA 4.4.** *Let  $(X, d)$  be a conical Robinson space with apex  $p$  with  $\delta < \text{diam}(X \setminus \{p\})$ , and  $<'$  be a compatible order on  $(X \setminus \{p\}, d)$  with minimal element  $x_*$  and maximal element  $x^*$ . A hole  $(y, z)$  is admissible if and only if  $d(x_*, y) \leq \delta$ ,  $d(z, x^*) \leq \delta$  and  $d(y, z) \geq \delta$ , and such a hole exists.*

*Proof.* Let  $(y, z)$  be a hole with  $d(x_*, y) \leq \delta$ ,  $d(z, x^*) \leq \delta$  and  $d(y, z) \geq \delta$ . Then for any  $x_* \leq' u <' v \leq' y$ ,  $d(u, v) \leq d(x_*, y) = \delta$  holds, for any  $z \leq' u <' v \leq' x^*$ ,  $d(u, v) \leq d(z, x^*) = \delta$  holds, and for any  $u \leq' y <' z \leq' v$ ,  $d(u, v) \geq d(y, z) \geq \delta$  holds. By [Lemma 4.2](#),  $(y, z)$  is an admissible hole. The reverse implication follows from [Lemma 4.2](#).

We prove the existence of such a hole. As  $x_*$  and  $x^*$  are extremal, we have  $d(x_*, x^*) = \text{diam}(X') = \text{diam}(X) > \delta$ . Note that the elements of  $X'$  are sorted by  $<'$  in increasing order of their distances from  $x_*$ , and in decreasing order of their distances to  $x^*$ . Thus the maximal element  $a$  such that  $d(a, x^*) > \delta$  and the minimal element  $b$  such that  $d(x_*, b) > \delta$  (relative to  $<'$ ) are well-defined. Let  $<$  be a compatible order for  $(X, d)$  with  $x_* < x^*$ . Note that  $p \in B_\delta(x_*) \cap B_\delta(x^*)$ . As  $d(x_*, x^*) > \delta$ , we obtain  $x_* < p < x^*$ .

**CLAIM 1.**  $X = B_\delta(x_*) \cup B_\delta(x^*)$ .

*Proof.* Suppose that there is a point  $w \in X$  with  $\min\{d(w, x_*), d(w, x^*)\} > \delta$ . We may assume that  $w < x_* < p < x^*$ . Then  $\delta = d(w, p) \geq d(w, x_*) > \delta$ , a contradiction.  $\square$

**CLAIM 2.**  $x_* \leq' a <' b \leq' x^*$ .

*Proof.* As  $b \notin B_\delta(x_*)$  and by [Claim 1](#),  $b \in B_\delta(x^*)$ , hence  $d(b, x^*) < d(a, x^*)$ , and thus  $x_* \leq' a <' b \leq' x^*$ .  $\square$

CLAIM 3. For any  $y \in X$  with  $y < p$ ,  $y \in B_\delta(x_*)$  holds. For any  $y \in X$  with  $y > p$ ,  $y \in B_\delta(x^*)$  holds.

*Proof.* If  $x_* \leq y < p$ , then  $d(x_*, y) \leq d(x_*, p) = \delta$  and  $y \in B_\delta(x_*)$ . If  $y < x_* < p < x^*$ , then  $d(y, x^*) \geq d(x_*, x^*) > \delta$ , implying that  $y \notin B_\delta(x^*)$ , and by Claim 1,  $y \in B_\delta(x_*)$ . The case when  $p < y$  is symmetric.  $\square$

Then  $a \notin B_\delta(x_*)$  implies that  $a < p$ , and similarly  $p < b$ . Thus there exist two consecutive elements  $y, z$  for  $<'$  with  $a \leq' y <' z \leq' b$  such that  $y < p < z$ . Then  $d(y, z) \geq d(y, p) = d(p, z) = \delta$ , and  $d(x_*, z) \leq \delta$ ,  $d(x, x^*) \leq \delta$  by the choice of  $a$  and  $b$ . Thus  $(y, z)$  is an admissible hole, concluding the proof of the lemma.  $\square$

From what precedes, we can derive the following:

PROPOSITION 4.5. Let  $(X, d)$  be a conical Robinson space with apex  $p$  and  $<'$  be a compatible order of  $X' := X \setminus \{p\}$ . Then Algorithm 4.1 computes an admissible hole  $(y, z)$  of  $X'$ ; if  $(y, z) = (x^*, x_*)$ , it returns  $X'$ , otherwise it returns the bipartition of  $X'$  into the points located before  $p$  and those located after  $p$  in  $<_{(y,z)}'$ . Algorithm 4.1 runs in  $O(|X'|)$ .

In the divide-and-conquer recognition algorithm of Robinson spaces we will apply Algorithm 4.1 with the  $p$ -copoints of  $X$  as  $X'$ .

---

**Algorithm 4.1** separateIfSeparable( $p, X'$ )

---

**Input:** a Robinson space  $(X, d)$  (implicit), conical with apex  $p$ , with  $X' = X \setminus \{p\}$  sorted along a compatible order  $<'$ .

**Output:** either  $X'$  if  $(x_*, x^*)$  is admissible, or the bipartition of  $X'$  into points before and after  $p$  in  $<_{(y,z)}'$  for some other admissible hole  $(y, z)$  otherwise. Each set is given with a representative.

```

1: let  $x_*, x^*$  be the minimum and maximum elements of  $(X', <')$ 
2: let  $\Delta = d(x_*, x^*)$ ,  $\delta = d(p, x_*)$ 
3: if  $\Delta \leq \delta$  then
4:   return  $[(x_*, X')]$ 
5: for  $y \in X' \setminus \{x^*\}$  in increasing compatible order do
6:   let  $z$  be the element consecutive to  $y$  in  $<'$ 
7:   if  $d(x_*, y) \leq \delta$  and  $d(z, x^*) \leq \delta$  and  $d(y, z) \geq \delta$  then
8:     return  $[(x_*, \{u \in X' : u \leq' y\}), (x^*, \{u \in X' : z \leq' u\})]$   $\triangleright (y, z)$  is an admissible hole
```

---

REMARK 1. When  $X'$  is not a  $p$ -copoint of a Robinson space, Algorithm 4.1 may return an arbitrary partition or nothing because the condition tested by the second **if** statement may never be satisfied. Thus when testing whether a dissimilarity space is Robinson, if the space is not Robinson, the algorithm may either fail in this procedure, or return an order that is not compatible.

**5. Mmodules in Robinson spaces.** In this section, we investigate the mmodules and the copoint partitions in Robinson spaces. We classify the copoints of  $\mathcal{C}_p$  into separable, non-separable, and tight. We show that if  $(X, d)$  is Robinson, then there exists a compatible order  $<$  in which all non-separable and tight copoints define intervals of  $<$ . Furthermore, we show that separable copoints define two intervals of  $<$ . Since each copoint of  $\mathcal{C}_p$  together with  $p$  define a conical subspace of  $(X, d)$ , applying Algorithm 4.1, we efficiently partition each separable copoint into two intervals. This leads to an extended copoint partition  $\mathcal{C}_p^*$  and to the extended quotient space  $(\mathcal{C}_p^*, d^*)$ , which is also a Robinson space. Finally, we show how to derive a compatible order  $<$  of  $(X, d)$  (satisfying the previous constraints) from compatible orders of the copoints of  $\mathcal{C}_p$  and a compatible order of the extended quotient  $(\mathcal{C}_p^*, d^*)$ . While the compatible orders of copoints are computed recursively, a compatible order of  $(\mathcal{C}_p^*, d^*)$  is computed via proximity orders, introduced and investigated in Section 6.

**5.1. Copoints in Robinson spaces.** Let  $\mathcal{C}_p = \{C_0 = \{p\}, C_1, \dots, C_k\}$  be a copoint partition with attaching point  $p$  of a Robinson space  $(X, d)$  (see Definition 3.7). For a copoint  $C_i$ , denote by  $\delta_i$  the distance  $d(p, x)$  for any point  $x \in C_i$  and we suppose that  $i < j$  implies that  $\delta_i \leq \delta_j$ . For  $C_i, C_j \in \mathcal{C}_p, i \neq j$ , let  $\delta_{ij}$  be the distance  $d(x, y)$  between any two points  $x \in C_i$  and  $y \in C_j$ . Since  $\mathcal{C}_p$  is a stable partition,  $\delta_{ij}$  is well-defined, moreover  $\delta_{ij}$  coincides with  $\hat{d}(C_i, C_j)$  in the quotient space  $(\mathcal{C}_p, \hat{d})$ . In this subsection, we investigate how in a compatible order  $<$  of  $(X, d)$  the copoints of  $\mathcal{C}_p$  compare to the point  $p$  and which

copoints of  $\mathcal{C}_p$  are not comparable to  $p$ . Notice that each subspace  $(C_i \cup \{p\}, d), i = 1, \dots, k$  is a cone over  $(C_i, d)$  with apex  $p$ . Applying [Lemma 4.4](#) to the Robinson space  $(C_i, d)$  and to the restriction  $<_i$  of  $<$  to  $C_i$ , we know that  $<_i$  admits at least one admissible hole. If this hole is defined by the rightmost and the leftmost points of  $<_i$ , then  $C_i$  is not divided in two parts, otherwise  $p$  divides  $C_i$  in two parts  $C_i^l$  and  $C_i^r$ . The next result shows that  $C_i^l$  and  $C_i^r$  are not only intervals of  $<_i$  but also of the global compatible order  $<$ :

**LEMMA 5.1.** *Let  $<$  be a compatible order of  $(X, d)$ ,  $p$  be a point of  $X$  and  $C_i \neq \{p\}$  be a copoint of  $\mathcal{C}_p$ . Then  $C_i^r := \{u \in C_i : p < u\}$  and  $C_i^l := \{u \in C_i : u < p\}$  are intervals of  $<$  (one of them may be empty).*

*Proof.* Let  $C_i^r$  contains at least 2 points (otherwise  $C_i^r$  is trivially an interval) and let  $x, z \in C_i^r$  be its minimum and maximum elements, respectively. Let  $y \in X$  with  $x < y < z$ , and let  $w \in X \setminus C_i$  such that either  $w < x$  or  $z < w$ . If  $w < x$ , then  $d(w, x) \leq d(w, y) \leq d(w, z) = d(w, x)$ , where the last equality comes from the fact that  $C_i$  is an mmodule that contains  $x, z$  but not  $w$ . Similarly, if  $z < w$ , then  $d(w, z) \leq d(w, y) \leq d(w, x) = d(w, z)$ . Hence for any such  $w$ , the distance  $d(w, y)$  is constant for any  $y \in [x, z]$ . As  $C_i$  is a maximal mmodule not containing  $p$ , this implies that  $[x, z] \subseteq C_i$ , that is  $C_i^r$  is an interval (possibly empty). Symmetrically,  $C_i^l$  is also an interval.  $\square$

**DEFINITION 5.2** (Classification of copoints). *A copoint  $C_i \in \mathcal{C}_p$  is separable if  $\text{diam}(C_i) > \delta_i$ , non-separable if  $\text{diam}(C_i) < \delta_i$ , and tight if  $\text{diam}(C_i) = \delta_i$ .*

**The running example.** In the running example, the copoints of  $\mathcal{C}_1$  are non-separable. The copoint  $C_3 = \{5, 12, 19\}$  of  $\mathcal{C}_2$  is separable and its two halved copoints are  $C_3^l = \{5, 19\}$  and  $C_3^r = \{12\}$ .

**LEMMA 5.3.** *If  $<$  is a compatible order of  $(X, d)$ , then any separable copoint  $C_i$  defines two intervals  $C_i^l$  and  $C_i^r$  of  $<$  such that  $d(x, y) \geq \delta_i$  for any  $x \in C_i^l, y \in C_i^r$  and  $\text{diam}(C_i^l) \leq \delta_i, \text{diam}(C_i^r) \leq \delta_i$ .*

*Proof.* Because  $\{p\} \cup C_i$  is conical with apex  $p$ , by [Lemma 4.4](#), in the restriction of  $<$  to  $C_i$ , the hole  $(y, z)$  with  $y < p < z$  is admissible. Then the two intervals from [Lemma 5.1](#) are  $C_i^l := \{x \in C_i : x \leq y\}$  and  $C_i^r := \{x \in C_i : x \geq z\}$ , and the result follows by [Lemma 4.2](#).  $\square$

**LEMMA 5.4.** *If  $<$  is a compatible order of  $(X, d)$ , then any non-separable copoint  $C_i$  defines a single interval of  $<$ .*

*Proof.* If  $x < p < y$  for  $x, y \in C_i$ , then  $d(x, y) \geq \max\{d(x, p), d(y, p)\} = \delta_i$ , which is impossible because  $d(x, y) \leq \text{diam}(C_i) < \delta_i$ . Thus  $C_i$  must be located on one side of  $p$ . By [Lemma 5.1](#),  $C_i$  is an interval of  $<$ .  $\square$

**LEMMA 5.5.** *Let  $C_i \in \mathcal{C}_p$  be a tight copoint of  $(X, d)$  and  $<$  be a compatible order such that the intervals  $C_i^r := \{u \in C_i : p < u\}$  and  $C_i^l := \{u \in C_i : u < p\}$  are not empty. Then the order  $<'$  defined by the rule:*

$$\text{for any } u < v, \text{ if } C_i^l < u < C_i^r \text{ and } v \in C_i^r, \text{ then } v <' u, \text{ otherwise } u <' v,$$

*is a compatible order of  $(X, d)$ . Consequently, if  $(X, d)$  is Robinson, then there exists a compatible order for which each tight copoint is a single interval.*

The order  $<'$  is thus obtained from  $<$  by moving  $C_i^r$  immediately after  $C_i^l$ . By symmetry, one could get a similar result by moving instead  $C_i^l$  in front of  $C_i^r$ .

*Proof.* Pick any three points  $x, y, z \in X$  such that they are not identically ordered by  $<$  and by  $<'$ . Then we can suppose without loss of generality that  $x \in C_i^r$  and  $C_i^l < y < C_i^r$ . Notice also that  $C_i^l \cup C_i^r$  is an interval of  $<'$ . We will show now that whatever is the order of  $x, y, z$  with respect to  $<$ , it does not contradict the compatibility of  $<'$ . First, let  $z \in X \setminus C_i$ . Let  $x_\ell$  be any point of  $C_i^l$ . Since  $C_i^l \cup C_i^r$  is an interval of  $<'$  containing  $x, x_\ell$  and not containing  $y, z$ , the order of  $x, y, z$  along  $<'$  is the same as the order of  $x_\ell, y, z$  along  $<'$ , which is the same as the order of  $x_\ell, y, z$  along  $<$ . Since  $d(x, z) = d(x_\ell, z), d(x, y) = d(x_\ell, y)$  and  $<$  is a compatible order, the result follows.

Now, let  $z \in C_i^r$ . Then we can suppose that  $y < x < z$  (the other case  $y < z < x$  is similar) and thus  $d(y, z) \geq \max\{d(y, x), d(x, z)\}$ . In this case, we have  $x <' z <' y$ . As  $d(y, z) = d(y, x)$ , we obtain  $d(y, x) \geq \max\{d(y, z), d(x, z)\}$  and we are done. Finally, let  $z \in C_i^l$ . Then we have  $z < y < x$ . If  $p < y$  or  $p = y$  (the case  $y < p$  is symmetric), then since  $C_i$  is a tight copoint, we obtain  $\delta_i = d(z, p) \leq d(z, y) = d(y, x) \leq d(z, x) = \delta_i$ . Consequently,  $d(y, x) = d(y, z) = d(x, z)$  and thus the triple  $x, y, z$  yields no contradiction for  $<'$ .  $\square$



|         | $C_0$ | $C_1$ | $C_2$ | $C_3'$ | $C_3''$ | $C_4$ | $C_5$ | $C_6$ |
|---------|-------|-------|-------|--------|---------|-------|-------|-------|
| $C_0$   | 0     | 10    | 8     | 2      | 2       | 11    | 5     | 1     |
| $C_1$   |       | 0     | 9     | 10     | 10      | 9     | 9     | 10    |
| $C_2$   |       |       | 0     | 8      | 8       | 9     | 6     | 8     |
| $C_3'$  |       |       |       | 0      | 3       | 11    | 5     | 2     |
| $C_3''$ |       |       |       |        | 0       | 11    | 5     | 2     |
| $C_4$   |       |       |       |        |         | 0     | 9     | 11    |
| $C_5$   |       |       |       |        |         |       | 0     | 5     |
| $C_6$   |       |       |       |        |         |       |       | 0     |

FIG. 5.1. The extended quotient space  $(\mathcal{C}_2^*, d^*)$ .

**Lemmas 5.3 to 5.5** imply the following result:

**PROPOSITION 5.6.** *If  $(X, d)$  is a Robinson space and  $\mathcal{C}_p$  is a copoint partition of  $X$ , then there exists a compatible order  $<$  in which each copoint  $C_i$  with  $\text{diam}(C_i) \leq \delta_i$  is an interval of  $<$  located either to the left or to the right of  $p$  and each copoint  $C_i$  with  $\text{diam}(C_i) > \delta_i$  defines two intervals  $C_i^l$  and  $C_i^r$  of  $<$  such that  $C_i^l < p < C_i^r$ .*

Next we consider only compatible orders of  $(X, d)$  satisfying the conditions of **Proposition 5.6**.

**5.2. Compatible orders from compatible orders of copoints and extended quotient.** Let  $\mathcal{C}_p = \{C_0 = \{p\}, C_1, \dots, C_k\}$  be a copoint partition with attaching point  $p$  of a Robinson space  $(X, d)$ . For a separable copoint  $C_i$  an *admissible bipartition* is a partition  $C_i$  into  $C_i^l$  and  $C_i^r$  such that  $\text{diam}(C_i^l) \leq \delta_i$ ,  $\text{diam}(C_i^r) \leq \delta_i$ , and  $d(x, y) \geq \delta_i$  for any  $x \in C_i^l$  and  $y \in C_i^r$ . This partition is defined by applying **Lemma 4.4** to each  $(C_i, d)$ ,  $i = 1, \dots, k$  and to each conical Robinson space  $(C_i \cup \{p\}, d)$ . Notice that  $C_i^l$  and  $C_i^r$  are no longer mmmodules because the distances between the points of  $C_i^l$  and  $C_i^r$  are not necessarily the same. We will call  $C_i^l$  and  $C_i^r$  *halved copoints*.

**DEFINITION 5.7** (Extended quotient). *Let  $\mathcal{C}_p^*$  denote the set of all non-separable and tight copoints of  $\mathcal{C}_p$  plus the set of all halved copoints corresponding to a choice of admissible bipartitions of each separable copoint. An extended quotient of  $(X, d)$  is the dissimilarity space  $(\mathcal{C}_p^*, d^*)$  defined in the following way: for  $i \neq j$ , the distance  $d^*(\alpha, \beta)$  between a pair of copoints or halved copoints  $\alpha, \beta$ , (1) is  $\delta_{ij}$  when one is indexed by  $i$  and the other is indexed by  $j$ , and (2) is  $\text{diam}(C_i)$ , the diameter of  $C_i$ , when  $\alpha$  and  $\beta$  are the two half copoints from the same copoint  $C_i$ . Notice that for any points  $u \in \alpha$  and  $v \in \beta$ , in the first case we have  $d^*(\alpha, \beta) = d(u, v)$  and in the second case, we have  $d^*(\alpha, \beta) \geq d(u, v)$ .*

**The running example.** The extended quotient space  $(\mathcal{C}_2^*, d^*)$  of the running example is given in **Figure 5.1**. It is unique as  $C_2$  has a unique admissible bipartition.

**LEMMA 5.8.** *If  $(X, d)$  is a Robinson space, then for any  $p \in X$ , its quotient  $(\mathcal{C}_p, \hat{d})$  (see **Definition 3.10**) and its extended quotient  $(\mathcal{C}_p^*, d^*)$  are Robinson spaces.*

*Proof.* It suffices to isometrically embed  $(\mathcal{C}_p, \hat{d})$  in  $(\mathcal{C}_p^*, d^*)$  and  $(\mathcal{C}_p^*, d^*)$  in  $(X, d)$ . The map  $\hat{\varphi} : \mathcal{C}_p \rightarrow \mathcal{C}_p^*$  maps any non-separable or tight copoint  $C_i$  to itself and any separable copoint  $C_i$  to one of its halves. From the definitions of  $(\mathcal{C}_p, \hat{d})$  and  $(\mathcal{C}_p^*, d^*)$ ,  $\hat{\varphi}$  is an isometric embedding. The map  $\varphi^* : \mathcal{C}_p^* \rightarrow X$  is defined as follows. We select one point  $x_i$  in each non-separable or tight copoint  $C_i$  and set  $\varphi^*(C_i) = x_i$  and we select a diametral pair  $\{x_i^l, x_i^r\}$  for each separable copoint  $C_i$  separated into  $C_i^l$  and  $C_i^r$  and set  $\varphi^*(C_i^l) = x_i^l, \varphi^*(C_i^r) = x_i^r$ . From the definition of  $(\mathcal{C}_p^*, d^*)$ ,  $\varphi^*$  is an isometric embedding.  $\square$

Now, we will show that if  $(X, d)$  is a Robinson space, then from any compatible order  $<^*$  of an extended quotient  $(\mathcal{C}_p^*, d^*)$  and from the compatible orders  $<_i$  of the copoints  $(C_i, d)$  of  $\mathcal{C}_p$ , we can define a compatible order  $<$  of  $(X, d)$ . We recall that any compatible order  $<_i$  of a separable copoint  $C_i$  has an admissible bipartition  $\{C_i^l, C_i^r\}$  of  $C_i$  as defined in **Subsection 4.2**. The total order  $<$  is defined as follows: for two points  $x, y$  of  $X$  we set  $x < y$  if and only if (1)  $x \in \alpha, y \in \beta$  for two different points  $\alpha, \beta \in \mathcal{C}_p^*$  and  $\alpha <^* \beta$  or (2)  $x, y \in \alpha \in \mathcal{C}_p^*$ ,  $\alpha \subseteq C_i$ , and  $x <_i y$ .



**PROPOSITION 5.9.** *If  $(X, d)$  is a Robinson space, then  $<$  is a compatible order of  $(X, d)$ .*

*Proof.* Pick any three distinct points  $u < v < w$  of  $X$  and let  $u \in \alpha, v \in \beta$ , and  $w \in \gamma$  with  $\alpha, \beta, \gamma \in \mathcal{C}_p^*$ . If  $\alpha = \beta = \gamma \subseteq C_i$ , then  $u <_i v <_i w$  and the result follows from the fact that  $<_i$  is a compatible order of  $(C_i, d)$ . Now, let  $\alpha, \beta$ , and  $\gamma$  be distinct. Then  $\alpha <^* \beta <^* \gamma$  and  $d^*(\alpha, \gamma) \geq \max\{d^*(\alpha, \beta), d^*(\beta, \gamma)\}$ . Since  $d^*(\alpha, \gamma) \geq d(u, w)$ ,  $d^*(\alpha, \beta) \geq d(u, v)$ , and  $d^*(\beta, \gamma) \geq d(v, w)$ , the inequality  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$  holds if  $d^*(\alpha, \gamma) = d(u, w)$ . Now suppose that say  $d^*(\alpha, \beta) > d(u, v)$ . From the definition of  $d^*$  this implies that  $\alpha$  and  $\gamma$  are the halved copoints  $C_i^l$  and  $C_i^r$  of  $C_i$  and say  $C_i^l <^* \beta \leq^* p <^* C_i^r$ . Then  $\beta$  belongs to a copoint  $C_j$  with  $\delta_j \leq \delta_i$ . Since  $u, w \in C_i$  and  $v \in C_j$ ,  $d(u, v) = d(v, w) = \delta_{ij} = d^*(\beta, \gamma) = d^*(\alpha, \beta) \leq \delta_i$ . On the other hand, since  $\{C_i^l, C_i^r\}$  is an admissible partition of  $C_i$ ,  $d(u, w) \geq \delta_i$  and we are done.

Finally, let  $\alpha = \beta$  or  $\beta = \gamma$ , say the first. First suppose that  $\alpha$  and  $\gamma$  are halved copoints of  $C_i$ :  $\alpha = C_i^l$  and  $\gamma = C_i^r$ . Then  $u <_i v <_i w$  and since  $<_i$  is a compatible order of  $(C_i, d)$ , we are done. Now suppose that  $\alpha$  and  $\gamma$  belong to different copoints, say  $\alpha \subseteq C_i$  and  $\gamma \subseteq C_j$ . Since  $u, v \in C_i$  and  $w \in C_j$ ,  $d(u, w) = d(v, w) = \delta_{ij}$ . It remains to prove that  $d(u, v) \leq \delta_{ij}$ . By [Proposition 5.6](#), there is a compatible order for which  $\alpha$  is an interval, in particular for which  $w$  is not between  $u$  and  $v$ . This implies  $d(u, v) \leq \max\{d(u, w), d(v, w)\} = \delta_{ij}$ . This concludes the proof of the proposition.  $\square$

**6. Proximity orders.** In this section, we introduce the concepts of  $p$ -proximity order and  $p$ -proximity pre-order for a compatible order of  $(X, d)$ . We show that  $p$ -proximity pre-orders can be efficiently computed by computing and ordering the copoints of  $p$ . For  $p$ -trivial Robinson spaces (see [Definition 3.8](#)), in particular for quotient spaces, the  $p$ -proximity pre-orders are  $p$ -proximity orders. Furthermore, we prove that extended quotients of Robinson spaces (even if they are not  $p$ -trivial) still admit  $p$ -proximity orders, which can be derived from  $p$ -proximity pre-orders. In all those algorithmic results, we compute a  $p$ -proximity pre-order or a  $p$ -proximity order without the knowledge of any compatible order. The main result of this section is the description of an efficient algorithm showing, given a  $p$ -proximity order  $<$ , how to compute a compatible order for which  $<$  is a  $p$ -proximity order. Applied to the extended quotients, this algorithm will be the merging step of our divide-and-conquer recognition algorithm described in [Section 7](#). Consequently, our recognition algorithm is recursively applied to each copoint individually, then the resulting orders of the copoints are merged into a single compatible order using the  $p$ -proximity order for the extended quotient. This way, we do not recurse on the extended quotient, which may coincide with the input space.

**6.1.  $p$ -Proximity orders.** We start with the definition of a  $p$ -proximity order.

**DEFINITION 6.1** ( $p$ -Proximity orders and pre-orders). *Let  $(X, d)$  be a Robinson space with a compatible order  $<$  and let  $p$  be a point of  $X$ . A  $p$ -proximity order for  $<$  is a total order  $<$  on  $X$  such that*

(PO1) *for all distinct  $x, y \in X$ , if  $x < y$ , then  $d(p, x) \leq d(p, y)$ ,*

(PO2) *for all distinct  $x, y \in X \setminus \{p\}$ ,  $x < y$  implies that either  $y < p$  and  $y < x$ , or  $y > p$  and  $y > x$  (i.e.  $y$  is not between  $p$  and  $x$  in the compatible order  $<$ ). Equivalently, for all  $x \in X \setminus \{p\}$ , the set  $\{t \in X : t < x\}$  is an interval for  $<$ .*

*A  $p$ -proximity pre-order for a compatible order  $<$  is a pre-order  $\leq$  on  $X$  which can be refined to a  $p$ -proximity order for  $<$ , and two distinct elements are equal only if they belong to the same copoint of  $\mathcal{C}_p$ .*

Notice that  $p$  is the minimum of  $<$  and that if  $d(p, x) < d(p, y)$ , then  $y < p$  and  $y < x$  or  $y > p$  and  $y > x$ . Intuitively, given a compatible order  $<$ , a  $p$ -proximity order  $<$  for  $<$  can be obtained by shuffling the elements smaller than  $p$  in reverse order into the elements larger than  $p$ , and adding  $p$  as the minimum.

For two disjoint sets  $S, S'$  of  $X$ , we denote  $S < S'$  when for each  $x \in S, y \in S'$ , either  $y < p$  and  $y < x$  or  $y > p$  and  $y > x$ . We denote  $S \leq_p S'$  when for each  $x \in S, y \in S', d(p, S) \leq d(p, S')$ .

One can build a  $p$ -proximity pre-order for a compatible order  $<$  without the knowledge of  $<$ . We will use [Algorithm 6.1](#), which is a variant of [Algorithm 2.1](#) (*partitionRefine*) and also uses [Algorithm 2.2](#) (*refine*). It differs from the stable partition algorithm in making a distinction between in-pivots (*In*) and out-pivots (*Out*), that are respectively smaller and bigger in the  $p$ -proximity order than the set  $S$  to refine. When using an out-pivot, the output of *refine* must be reordered, on [Line 7](#). Notice that when  $m = 1$ , that is when the call to *refine* on [Line 4](#) returns a single part, the algorithm calls itself on [Line 13](#) with the same parameters except that  $q$  is removed from the sets of pivots.

**LEMMA 6.2.** *Let  $(X, d)$  be a Robinson space. Let  $In, S$  and  $Out$  be disjoint subsets of  $X$ , and  $p \in X \setminus S$ . Suppose that the following preconditions hold:*

**Algorithm 6.1** recursiveRefine( $p, In, S, Out$ )

**Input:** a Robinson space  $(X, d)$  (implicit), a point  $p \in X$ , a set  $S \subseteq X$ , two disjoint subsets  $In, Out \subseteq X \setminus S$  of inner-pivots and outer-pivots given as lists.

**Output:** an ordered partition  $[S_1^\star, S_2^\star, \dots, S_k^\star]$  of  $S \setminus \{p\}$  (encoding a partial  $p$ -proximity pre-order).

```

1: if  $In \cup Out = \emptyset$  then
2:   return  $[S]$ 
3: let  $q \in In \cup Out$   $\triangleright$  choose  $q$  to be the first element of  $In$  or  $Out$ 
4: let  $[S_1, \dots, S_m] = \text{refine}(q, S)$ 
5: if  $q \in Out$  then
6:   let  $\alpha = \min(\{j \in \{1, \dots, m\} : d(S_j, q) > d(p, q)\} \cup \{m+1\})$ 
7:   let  $[S'_1, \dots, S'_m] = [S_{\alpha-1}, S_{\alpha-2}, \dots, S_1, S_\alpha, S_{\alpha+1}, \dots, S_m]$ 
8: else
9:   let  $[S'_1, \dots, S'_m] = [S_1, \dots, S_m]$ 
10: for  $i \in \{1, \dots, m\}$  do
11:   let  $In_i = \text{concatenate}(S'_1, \dots, S'_{i-1}, In \setminus \{q\})$ 
12:   let  $Out_i = \text{concatenate}(S'_{i+1}, \dots, S'_m, Out \setminus \{q\})$ 
13:   let  $T_i = \text{recursiveRefine}(p, In_i, S'_i, Out_i)$ 
14: return  $\text{concatenate}(T_1, \dots, T_m)$ 

```

(i)  $In \leq_p S \leq_p Out$ ,  
(ii)  $In < S < Out$ ,  
(iii) for all  $x, x' \in S$  and  $y \in X \setminus (In \cup S \cup Out)$ ,  $d(x, y) = d(x', y)$ ,  
and either  $(In, Out) = (\{p\}, \emptyset)$  or  $p \notin In \cup S \cup Out$ . Let  $[S_1^\star, \dots, S_k^\star]$  be the output of [Algorithm 6.1](#) with input  $(p, In, S, Out)$ . Then the following postconditions hold:

- (iv)  $\{S_1^\star, \dots, S_k^\star\}$  is a partition of  $S$  into mmodules,  
(v)  $S_1^\star < S_2^\star < \dots < S_k^\star$ .

*Proof.* We prove by induction on the call tree that if at each recursive call, the preconditions (i), (ii) and (iii) hold, then the postconditions (iv) and (v) also hold. In the base case, that is when [Algorithm 6.1](#) returns at Line 2, (iv) follows from (iii) ( $S$  is an mmodule), and (v) is trivial. We assume now that it returns at Line 14 and we will use the notation of the algorithm.

In order to use the induction hypothesis, we prove that the preconditions (i), (ii), (iii) hold for each recursive call at Line 13. Let  $i \in \{1, \dots, m\}$ . Notice that by construction  $In_i, S'_i$  and  $Out_i$  are disjoint and  $p \notin In_i \cup S'_i \cup Out_i$ . If  $In = \{p\}$  and  $Out = \emptyset$ , then by [Lemma 2.5](#) applied to the call to *refine*,  $S'_1 \leq_p S'_2 \leq \dots \leq S'_m$ , from which (i) follows for  $i$ . Moreover in that case, for any  $x \in S'_j$  and  $y \in S'_{j'}$  with  $j < j'$ ,  $d(p, x) < d(p, y)$  implies that either  $y < p$  and  $y < x$  or  $y > p$  and  $y > x$ , thus  $S'_j < S'_{j'}$ , proving (ii) for each  $i$ . Otherwise,  $p \notin In \cup S \cup Out$ , thus by (iii),  $d(p, x) = d(p, x')$  for any  $x, x' \in S$ , also proving (i) for  $i$ . Condition (ii) follows from the next claim.

CLAIM 4. If  $p \neq q$ , then for each  $i < j$ ,  $S'_i < S'_j$ .

*Proof.* We may assume that  $q < p$  (the case  $p < q$  is analogous). If  $q \in In$ , then for any  $y \in S$  we have  $y < q < p < y$  or  $q < p < y$  by (ii). Let  $x \in S'_i$  and  $y \in S'_j$ . By [Lemma 2.5](#),  $d(q, x) < d(q, y)$ . Hence we have either  $y < x < q < p$ , or  $y < q < p < x$ , or  $x < q < p < y$ , or  $q < p < x < y$ . In any of these cases,  $y$  is not between  $p$  and  $x$ , hence  $S'_i < S'_j$ .

If  $q \in Out$ , then for any  $y \in S$ , we have  $q < y < p$  or  $q < p < y$  by (ii). Moreover if  $d(y, q) > d(p, q)$  then  $q < p < y$ , and for any  $x \in S$  with  $d(x, q) < d(y, q)$ ,  $q < x < y$ . Let  $x \in S'_i$  and  $y \in S'_j$ , with  $i < j$ . If  $i < \alpha \leq j$ , then  $d(q, x) \leq d(q, p) < d(q, y)$  and  $q < x < p < y$  or  $q < p < x < y$ . If  $i < j < \alpha$  then  $d(q, y) < d(q, x) \leq d(q, p)$  and  $q < y < x < p$  or  $q < y < p < x$ . If  $\alpha \leq i < j$ , then  $d(q, p) < d(q, x) < d(q, y)$  and  $q < p < x < y$ . In every case,  $y$  is not between  $p$  and  $x$ , hence  $S'_i < S'_j$ .  $\square$

Let  $x, x' \in S'_i$ . By (iii) on the input, for each  $y \in X \setminus (In \cup S \cup Out)$ , we have  $d(x, y) = d(x', y)$ . Moreover, by [Lemma 2.5](#), for all  $x, x' \in S'_i$ , we have  $d(q, x) = d(q, x')$ . Hence (iii) for  $i$  follows by observing that  $X \setminus (In_i, S'_i, Out_i) = X \setminus (In \cup S \cup Out_i) \cup \{q\}$ .

Having proved that all the preconditions (i), (ii) and (iii) hold for each recursive call, we get by induction

that for each  $i$ , the postconditions (iv) and (v) hold. Consequently,  $T_i$  is a partition of  $S'_i$  into mmodules sorted by  $<$ . We now show that (iv) and (v) also hold for  $S_1^*, \dots, S_k^*$ . From Line 14 and because  $\{S_1', \dots, S_m'\}$  is a partition of  $S$  into mmodules, we obtain that (iv) holds. Finally, for any  $S_i^*$  and  $S_j^*$  with  $i < j$ , either there is  $k$  with  $S_i^*, S_j^* \subset S'_k$ , in which case by (v) for recursive call  $k$ ,  $S_i^* < S_j^*$ , or there are  $k < k'$  with  $S_i^* \in S'_k$  and  $S_j^* \in S'_{k'}$ , in which case  $S'_k < S'_{k'}$  by Claim 4, hence  $S_i^* < S_j^*$ , proving (v).  $\square$

**PROPOSITION 6.3.** *Let  $(X, d)$  be a Robinson space and let  $p$  be any point of  $X$ . Then Algorithm 6.1 with input  $(p, \{p\}, X \setminus \{p\}, \emptyset)$  returns the copoint partition  $\mathcal{C}_p = (C_0 = \{p\}, C_1, \dots, C_k)$  of  $p$  sorted along a  $p$ -proximity pre-order  $<$  for any (unknown) compatible order  $<$  of  $(X, d)$ .*

*Proof.* We can apply Lemma 6.2 on the initial call to Algorithm 6.1, from which it only remains to prove that each mmodule  $S_i^*$  is a  $p$ -copoint. Suppose it is not a copoint, then it is contained in a copoint  $C \subset S$ . Consider the deepest recursive call of Algorithm 6.1 for which  $C \subseteq S$ . Then it must be that on Line 4, *refine* splits  $C$  into several parts, which means that there is  $x, y \in C$  such that  $d(q, x) \neq d(q, y)$  by Lemma 2.5, for  $q \notin C$ . But this contradicts the fact that  $C$  is an mmodule.  $\square$

Now, we analyse the complexity of Algorithm 6.1 without counting the time spent by all the recursive calls to *refine*. This will be done at a later stage in our analysis.

**LEMMA 6.4.** *Without counting the time spent in the calls to Algorithm 2.2, Algorithm 6.1 with input  $(p, [q], S, [])$  and output  $[S_1^*, \dots, S_k^*]$  runs in time  $O(|S|^2 - \sum_{j=1}^k |S_j^*|^2)$ .*

*Proof.* We consider the tree of recursive calls, where each node correspond to some subset of  $S$ , and the leaves correspond to  $[S_1^*, S_2^*, \dots, S_k^*]$ . Firstly we bound the number of nodes in that tree. To this end, notice that for each leaf  $S_j^*$ , for  $j' \neq j$ , each element  $x \in S_j^*$  has been added to either  $In_i$  or  $Out_i$  at the recursive call corresponding to the lowest common ancestor of  $S_j^*$  and  $S_{j'}^*$ . Because each recursive call removes exactly one element from  $In_i \cup Out_i$ , the depth of  $S_j^*$  is  $|S \setminus S_j^*|$ . Thus the number of nodes is at most  $C := \sum_{j=1}^k |S_j^*| |S \setminus S_j^*| = |S|^2 - \sum_{j=1}^k |S_j^*|^2$ .

Then, the sum of all  $m$  (defined on line 4) over all recursive calls is the sum of arities of the nodes of the tree, hence is less than  $C$ . This implies that the operations on lines 6, 7 and 14, as well as the cost of constant-time operations of each call, contribute  $O(C)$  to the total cost of the algorithm. It remains to bound the cost of lines 11 and 12. But the cost of inserting elements into  $In_i$  and  $Out_i$  can be no more than the cost of removing all those inserted elements, which we have already bounded by  $C$ .  $\square$

For  $p$ -trivial Robinson spaces, the  $p$ -proximity pre-orders are orders:

**PROPOSITION 6.5.** *If  $(X, d)$  is a  $p$ -trivial Robinson space, then any  $p$ -proximity pre-order is a total order and it can be computed in  $O(|X|^2)$  time.*

*Proof.* Since all copoints of  $\mathcal{C}_p$  are trivial, a  $p$ -proximity pre-order  $\leq$  is by definition a total order, hence  $\leq$  is a  $p$ -proximity order. By Proposition 6.3, this  $p$ -proximity order can be computed in  $O(|X|^2 - |X| + 1)$  time because  $k = |X| - 1$  and all copoints have size 1.  $\square$

**The running example.** First consider Algorithm 6.1 for  $p = 1$  on input  $(p, \{p\}, X \setminus \{p\}, \emptyset)$ . Then on Line 4, using  $q = p = 1$ , we get a  $p$ -proximity pre-order encoded by the ordered partition

$$[\{9, 17\}, \{6, 10\}, \{3, 4, 7, 8, 11, 13, 14, 16, 18\}, \{2, 5, 12, 15, 19\}].$$

Since  $q \in In$ , the pre-order is not modified and the recursive calls output respectively  $[\{17\}, \{9\}]$ ,  $[\{10\}, \{6\}]$ ,  $[\{3, 4, 8, 16, 18\}, \{11, 13, 14\}, \{7\}]$  and  $[\{2, 5, 12, 15, 19\}]$ . Thus the  $p$ -proximity pre-order is:

$$[\{1\}, \{17\}, \{9\}, \{10\}, \{6\}, \{3, 4, 8, 16, 18\}, \{11, 13, 14\}, \{7\}, \{2, 5, 12, 15, 19\}].$$

To go deeper, let us detail the third recursive call, with parameters

$$(p, In = \{9, 17, 6, 10\}, S = \{3, 4, 7, 8, 11, 13, 14, 16, 18\}, Out = \{2, 5, 12, 15, 19\}).$$

For any element in  $In$ , no refinement happens ( $m = 1$ ), hence there are multiple recursive calls until getting to parameters  $(p, In = \{1\}, S = \{3, 4, 7, 8, 11, 13, 14, 16, 18\}, Out = \{2, 5, 12, 15, 19\})$ . Then when choosing  $q = 2$  from  $Out$ , the call to *refine* on Line 4 returns a non-trivial ordered partition  $[\{11, 13\}, \{3, 4, 8, 16, 18\}, \{7\}]$ ,

where  $d(2, 11) = 5$ ,  $d(2, 3) = 8$ ,  $d(2, 7) = 11$  and  $d(p, 2) = 10$ . Then after Line 7, the order is changed to  $[\{3, 4, 8, 16, 18\}, \{11, 13\}, \{7\}]$ . As each of these parts is a copoint, they will not get subdivided further in subsequent recursive calls, and this  $p$ -proximity order is returned.

**6.2.  $p$ -Proximity orders for extended quotients.** Let  $(X, d)$  be a Robinson space and  $p \in X$ . By Lemma 3.11,  $(\mathcal{C}_p, \hat{d})$  is  $p$ -trivial. By Lemma 5.8,  $(\mathcal{C}_p, \hat{d})$  and  $(\mathcal{C}_p^*, d^*)$  are Robinson, but  $(\mathcal{C}_p^*, d^*)$  is not  $p$ -trivial. Nevertheless,  $(\mathcal{C}_p^*, d^*)$  has a  $p$ -proximity order:

PROPOSITION 6.6. *For an extended quotient  $(\mathcal{C}_p^*, d^*)$  of a Robinson space  $(X, d)$  one can compute a  $p$ -proximity order  $<$  for some (unknown) compatible order  $<^*$  of  $(\mathcal{C}_p^*, d^*)$ .*

*Proof.* By Lemma 5.8,  $(\mathcal{C}_p^*, d^*)$  is Robinson. By Proposition 6.3 there is a  $p$ -proximity pre-order  $\preceq$  for a compatible order  $<$  of  $(\mathcal{C}_p^*, d^*)$ . One can easily see that the copoints of  $p$  in  $(\mathcal{C}_p^*, d^*)$  are either trivial or of the form  $(C_i', C_i'')$  for a separable copoint  $C_i \in \mathcal{C}_p$ . We refine  $\preceq$  into an order  $<$  by arbitrarily ordering each such pair  $\{C_i', C_i''\}$  with  $C_i' < C_i''$ . We assert that there exists a compatible order  $<^*$  on  $\mathcal{C}_p^*$  having  $<$  as a  $p$ -proximity order. By Lemma 5.8,  $(\mathcal{C}_p^*, d^*)$  is isometric to the restriction of  $(X, d)$  to the following set: the point  $p$ , one representative  $x_i$  for each tight or non-separable copoint  $C_i$ , and a diametral pair  $\{x_i', x_i''\}$  of  $C_i$  as representatives for each separable copoint  $C_i$  (we then have  $x_i' < p < x_i''$  or  $x_i'' < p < x_i'$ ).

We define  $<^*$  from  $<$  by permuting the representatives  $(x_i', x_i'')$  of each separable copoint  $C_i$  with  $x_i'' < x_i'$ , so that  $x_i' <^* x_i''$  (in this case,  $x_i'$  and  $x_i''$  are said to be *permuted*). Then,  $<^*$  is also a compatible order: indeed the dissimilarity matrices with rows and columns ordered by  $<$  and  $<^*$  are identical, because  $\{x_i', x_i''\}$  is an mmodule of  $\mathcal{C}_p^*$ . Next we prove that  $<$  is a  $p$ -proximity order for  $<^*$ . To prove (PO1), as  $<$  is a refinement of  $\preceq$ , we only need to prove that  $d(p, x_i') \leq d(p, x_i'')$  for any separable copoint. This is indeed so because  $d(p, x_i') = d(p, x_i'')$ . To prove (PO2), we first prove that  $\preceq$  is a pre-order for  $<^*$ .

We distinguish four cases. First, suppose that  $x \not\preceq y$  and neither  $x$  nor  $y$  was permuted with each other or another point. Then the relative order of  $p$ ,  $x$  and  $y$  does not change, hence in  $<^*$ ,  $y$  is not between  $p$  and  $x$ . Second, suppose that  $x \not\preceq y$ ,  $x$  was permuted as member of a pair  $(x_i', x_i'')$ , and  $y$  was not permuted. This means that  $x_i'' < p < x_i' < y$  (because  $\preceq$  is a  $p$ -proximity pre-order), and after permuting we have  $x_i' <^* p <^* x_i'' <^* y$ , hence  $y$  is not between  $x_i'$  and  $p$  nor between  $p$  and  $x_i''$  in  $<^*$ . Now, suppose that  $x \not\preceq y$  and  $x$  was not permuted, but  $y$  was permuted as member of a pair  $(y_j', y_j'')$ . This is similar to the previous case:  $y_j'' < p < x < y_j'$  or  $y_j'' < x < p < y_j'$ , and after permutation, we have  $y_j' <^* p <^* x <^* y_j''$  or  $y_j' <^* x <^* p <^* y_j''$ . Anyways we get that  $y$  is not between  $p$  and  $x$  in  $<^*$ . Finally, suppose that  $x \not\preceq y$  and both  $x, y$  were permuted. Hence they belong to pairs  $(x_i', x_i'')$  and  $(y_j', y_j'')$ , respectively. Then  $y_j'' < x_i'' < p < x_i' < y_j'$  and  $y_j' <^* x_i' <^* p <^* x_i'' <^* y_j''$ . Hence  $y$  is not between  $p$  and  $x$  in  $<^*$ . Consequently, we proved that  $\preceq$  is a pre-order for  $<^*$ .

Since  $<$  is a refinement of  $\preceq$ , it suffices to prove (PO2) for the pairs  $(x_i', x_i'')$ . But  $x_i' <^* p <^* x_i''$ , hence  $x_i''$  is not between  $p$  and  $x_i'$ , concluding the proof that  $<$  is a  $p$ -proximity order for  $<^*$ .  $\square$

**6.3. Compatible orders from  $p$ -proximity orders.** From the definition of a  $p$ -proximity order  $<$  for a compatible order  $<$ , it follows that  $<$  can be recovered provided  $<$  is given and we know which elements of  $X \setminus \{p\}$  are located to the left and to the right of  $p$ . This is specified by the following result:

LEMMA 6.7. *Let  $(X, d)$  be a Robinson space with a compatible order  $<$ . Then  $<$  is fully determined by a  $p$ -proximity order  $<$  for  $<$  and the bipartition  $(L, R) = (\{x \in X : x < p\}, \{x \in X : x > p\})$ . More precisely, for any  $u, v \in X \setminus \{p\}$ , we have  $u < v$  if and only if*

- (i) either  $u \in L$ ,  $v \in R$ ,
- (ii) or  $u, v \in L$ , and  $v < u$ ,
- (iii) or  $u, v \in R$  and  $u < v$ .

For a point  $u \in X \setminus \{p\}$ , we denote by  $\text{side}(u)$  the set  $L$  or  $R$  to which  $u$  belongs and by  $\text{opp}(u)$  the other set. Then we have the following elementary result:

LEMMA 6.8. *Let  $(X, d)$  be a Robinson space,  $p$  a point of  $X$ ,  $<$  a  $p$ -proximity order on  $(X, d)$  and  $u, v \in X \setminus \{p\}$  such that  $u < v$ . Then*

- (i)  $d(u, v) < d(p, v)$  implies that  $\text{side}(u) = \text{side}(v)$ ,
- (ii)  $d(u, v) > d(p, v)$  implies that  $\text{side}(u) = \text{opp}(v)$ .

It remains to construct the sets  $L$  and  $R$ ; this is done by [Algorithm 6.2](#). To explain how this algorithm works we will use two graphs  $G$  and  $H$ ; they are not explicitly used by [Algorithm 6.2](#) but are used in the proof of its correctness. The graph  $G$  has  $X \setminus \{p\}$  as the vertex-set and the edges are defined in such a way that the side of each vertex in a connected component of  $G$  is uniquely defined by fixing arbitrarily the side of any vertex from that component. The second graph  $H$  has the connected components of  $G$  plus  $\{p\}$  as vertices and the pairs of components which are “tangled” with respect to  $<$  as edges. Again, fixing the side of an arbitrary point from a connected component of  $H$ , uniquely defines the side of all points of  $X \setminus \{p\}$  belonging to that connected component. Finally, we prove that if we order the connected components  $\mathcal{K}_0 = \{\{p\}\}, \mathcal{K}_1, \dots, \mathcal{K}_s$  of  $H$  by their maximal elements  $m_1, \dots, m_s$  in the order  $<$ , then for any  $i = 1, \dots, s-1$ , the union  $\bigcup_{j=0}^i \mathcal{K}_j$  is an mmodule of  $(X, d)$  and an interval of  $<$ . This implies that the sides of points in each  $\mathcal{K}_i$  can be determined independently of the sides of the points from  $\mathcal{K}_{i+1}, \dots, \mathcal{K}_s$ .

The graph  $G$  has  $X \setminus \{p\}$  as the vertex-set and  $\{uv : u < v \wedge d(u, v) \neq d(p, v)\}$  as the edge-set. Denote by  $\mathcal{C}$  the set of connected components of the graph  $G$ .

**LEMMA 6.9.** *For any connected component  $C$  of the graph  $G$ , the sides of all points  $C$  are uniquely determined by fixing arbitrarily the side of any vertex  $r$  of  $C$ .*

*Proof.* Let  $T$  be any spanning tree of  $C$  and suppose that  $T$  is rooted at the vertex  $r$ . Suppose also that  $\text{side}(r)$  was fixed. Then we proceed by induction on the length of the unique path of  $T$  connecting  $r$  with any vertex  $v \in C$ . Let  $uv$  be the edge of  $T$  on this path; whence,  $\text{side}(u)$  was already determined. Since  $uv$  is an edge of  $G$ , either we have  $u < v$  and  $d(u, v) \neq d(p, v)$  or  $v < u$  and  $d(u, v) \neq d(p, u)$ . In both cases,  $\text{side}(v)$  is well-defined by applying [Lemma 6.8](#).  $\square$

By [Lemma 6.9](#), if  $G$  is connected, then the sides of all vertices of  $G$  are well-defined and one can retrieve the sets  $L$  and  $R$  up to symmetry. Now suppose that  $G$  is not connected. We say that two connected components  $C, C' \in \mathcal{C}$  of  $G$  are *tangled* if  $C$  and  $C'$  are not comparable by  $<$ , i.e., either there exist  $x, y \in C$  and  $z \in C'$  such that  $x < z < y$  holds or there exist  $x, y \in C'$  and  $z \in C$  such that  $x < z < y$  holds.

**LEMMA 6.10.** *If  $C, C' \in \mathcal{C}$  and  $C, C'$  is a tangled pair for which there exist  $x, y \in C$  and  $z \in C'$  such that  $x < z < y$ , then there exist  $x', y' \in C$  such that  $x' < z < y'$  and  $x'y'$  is an edge of  $G$ .*

*Proof.* Since  $x, y \in C$  there exists a path  $P = (x = x_1, x_2, \dots, x_{t-1}, x_t = y)$  connecting the vertices  $x$  and  $y$  in  $C$ . Since  $x < z < y$ , necessarily  $P$  contains an edge  $x_i x_{i+1}$  such that  $x_i < z < x_{i+1}$ . Thus we can set  $x' = x_i$  and  $y' = x_{i+1}$ .  $\square$

**LEMMA 6.11.** *Let  $x < z < y$ ,  $xy$  be an edge of  $G$ , and suppose that  $yz$  is not an edge of  $G$ . If  $d(x, y) < d(p, y)$ , then  $\text{side}(x) = \text{side}(y) \neq \text{side}(z)$  and if  $d(x, y) > d(p, y)$ , then  $\text{side}(x) \neq \text{side}(y) = \text{side}(z)$ .*

*Proof.* Since  $x < y$  and  $xy$  is an edge of  $G$ , we have  $d(x, y) \neq d(p, y)$ . On the other hand, since  $zy$  is not an edge of  $G$ , we have  $d(z, y) = d(p, y)$ .

First case:  $d(x, y) < d(p, y)$ . By [Lemma 6.8](#), we have  $\text{side}(x) = \text{side}(y)$ . Suppose that  $\text{side}(z) = \text{opp}(y)$  in a compatible order  $<$  for which  $<$  is a  $p$ -proximity order. Since  $x < z < y$ , by [Lemma 6.7](#)  $z$  must be located between  $x$  and  $y$  in  $<$ . Consequently,  $d(z, y) = d(p, y) > d(x, y)$ , contrary to the fact that in  $<$  the point  $z$  is located between  $x$  and  $y$ . This contradiction shows that  $\text{side}(z) \neq \text{side}(x) = \text{side}(y)$  in this case.

Second case:  $d(x, y) > d(p, y)$ . By [Lemma 6.8](#), we have  $\text{side}(x) \neq \text{side}(y)$ . Suppose that  $\text{side}(z) = \text{side}(x)$ . Since  $x < z$ , by [Lemma 6.7](#)  $x$  must be located between  $z$  and  $p$  in any compatible order  $<$  for which  $<$  is a  $p$ -proximity order. Since  $\text{side}(z) = \text{side}(x) = \text{opp}(y)$ ,  $x$  is also located between  $z$  and  $y$  in  $<$ . Consequently,  $d(x, y) > d(p, y) = d(z, y)$ , which is impossible because in  $<$  the point  $x$  is located between  $z$  and  $y$ . This contradiction shows that  $\text{side}(z) = \text{side}(y)$  in this case.  $\square$

**LEMMA 6.12.** *If  $C, C' \in \mathcal{C}$  and  $C, C'$  is a tangled pair, then the sides of all points of  $C'$  are uniquely determined by the sides of points of  $C$  and, vice-versa, the sides of all points of  $C$  are uniquely determined by the sides of points of  $C'$ .*

*Proof.* Since  $C, C'$  is a tangled pair, either there exist  $x, y \in C$  and  $z \in C'$  such that  $x < z < y$  or there exist  $x, y \in C'$  and  $z \in C$  such that  $x < z < y$ , say the first. By [Lemma 6.10](#), there exists an edge  $x'y'$  of  $C$  such that  $x' < z < y'$ . By [Lemma 6.11](#), the side of  $z$  is uniquely determined by the sides of  $x'$  and  $y'$  (and thus the sides of  $x'$  and  $y'$  are uniquely determined by the side of  $z$ ). By [Lemma 6.9](#) the sides of points of  $C'$  are uniquely determined by the side of  $z$ .  $\square$



The graph  $H$  has the set  $\mathcal{C}$  of connected components of  $G$  as vertices and the tangled pairs of  $\mathcal{C}$  as edges. Let  $\Sigma = \{\mathcal{K}_1, \dots, \mathcal{K}_s\}$  be the connected components of the graph  $H$ . Set also  $\mathcal{K}_0 := \{p\}$ . We will denote by  $\mathcal{K}_i^\oplus$  the set of all points of  $X$  belonging to the connected components of  $G$  included in  $\mathcal{K}_i$  and by  $\mathcal{K}_{\leq i}^\oplus$  the union of the sets  $\mathcal{K}_0^\oplus, \mathcal{K}_1^\oplus, \dots, \mathcal{K}_i^\oplus$ . For each  $\mathcal{K}_i, i = 1, \dots, s$  we denote by  $m_i$  the maximal point of  $\mathcal{K}_i^\oplus$  in  $<$ . Let also  $C(m_i) \in \mathcal{C}$  be the component of  $G$  containing the point  $m_i$ . Suppose that the connected components of  $H$  are ordered  $\{\mathcal{K}_1, \dots, \mathcal{K}_s\}$  according to the order of the points  $m_1, \dots, m_s$  in  $<$ :  $p < m_1 < \dots < m_s$ .

LEMMA 6.13. *For any  $i = 1, \dots, s$ ,  $\mathcal{K}_i^\oplus = \{x \in X \setminus \{p\} : m_{i-1} < x \leq m_i\}$  holds and the interval  $\{x \in X : x \leq m_i\} = \mathcal{K}_{\leq i}^\oplus$  is an mmodule of  $(X, d)$ .*

*Proof.* It suffices to establish the lemma for the last component  $\mathcal{K}_s$  and use induction. Let  $u_*$  be the minimal element of  $\mathcal{K}_s^\oplus$  for  $<$ . By definition,  $m_{s-1}$  is the maximal element of  $\mathcal{K}_{\leq s-1}^\oplus$ . Therefore to prove that  $\mathcal{K}_s^\oplus = \{x \in X \setminus \{p\} : m_{s-1} < x \leq m_s\}$  it suffices to prove that  $m_{s-1} < u_*$ . Suppose, by a way of contradiction, that  $m_{s-1} > u_*$ . Let  $C'$  be the connected component of  $G$  containing  $u_*$ . Since  $C(m_s)$  and  $C'$  belong to  $\mathcal{K}_s$ , there exists a path  $C' = C_0, C_1, \dots, C_{k-1}, C_k = C(m_s)$  connecting  $C'$  and  $C(m_s)$  in the graph  $H$ . Let  $w_i, t_i$  be respectively the minimal and the maximal elements of  $C_i$  with respect to  $<$ . Since for all  $0 \leq i < k$ ,  $C_i C_{i+1}$  are tangled pairs, in  $<$  the intervals  $[w_i, t_i]$  and  $[w_{i+1}, t_{i+1}]$  have a nonempty intersection. Since  $w_0 < m_{s-1} < t_s$ , necessarily there exists an index  $i$  such that  $w_i < m_{s-1} < t_i$ . This implies that the connected components  $C(m_{s-1})$  and  $C_i$  are tangled. By the definition of the graph  $H$ ,  $C(m_{s-1})$  must belong to  $\mathcal{K}_s$ , a contradiction with the definition of the point  $m_{s-1}$ . This shows that  $m_{s-1} < u_*$  and establishes the first assertion for  $\mathcal{K}_s$ . Applying induction we conclude that  $\mathcal{K}_i^\oplus = \{x \in X \setminus \{p\} : m_{i-1} < x \leq m_i\}$  holds for any  $i = 1, \dots, s$ . This also implies that  $\mathcal{K}_{\leq i}^\oplus = \{x \in X : x \leq m_i\}$  holds for all  $i = 1, \dots, s$ .

To prove that  $\mathcal{K}_{\leq i}^\oplus$  is an mmodule, again we will prove the assertion for  $i = s - 1$  and use induction to get the result for all  $i$ . Let  $M = \{x \in X : x \leq m_{s-1}\}$ . Let  $x, y \in M$  and  $q \notin M$ . Then  $x < q$  and  $y < q$ . Since  $x$  and  $y$  do not belong to the connected component of  $G$  containing  $q$ , we also have  $d(p, q) = d(q, x)$  and  $d(p, q) = d(q, y)$ . Hence  $d(q, x) = d(q, y)$ , proving that  $M$  is an mmodule.  $\square$

From Lemma 6.10 it follows that a bipartition  $(L_i, R_i)$  of points of each connected component  $\mathcal{K}_i$  of the graph  $H$  is uniquely determined once the side of one of its points, say of  $m_i$ , is arbitrarily fixed. Let  $L = \bigcup_{i=1}^s L_i$  and  $R = \bigcup_{i=1}^s R_i$ . Now, we prove that the total order obtained from the bipartition  $(L, R)$ , where  $L$  and  $R$  are ordered according to the  $p$ -proximity order  $<$  (see Lemma 6.7) is a compatible order.

PROPOSITION 6.14. *Let  $(X, d)$  be a Robinson space and  $<$  be a  $p$ -proximity order. Let  $<'$  be a total order obtained from the bipartition  $(L, R)$  according to  $<$ , where  $L = \bigcup_{i=1}^s L_i, R = \bigcup_{i=1}^s R_i$  and  $(L_i, R_i)$  is the bipartition of  $\mathcal{K}_i^\oplus$  obtained by arbitrarily fixing  $\text{side}(m_i) \in \{L, R\}$ . Then  $<'$  is a compatible order of  $(X, d)$ .*

*Conversely, any compatible order  $<$  of  $(X, d)$  such that  $<$  is a  $p$ -proximity order for  $<$  is obtained in this way. Consequently,  $<$  is a  $p$ -proximity order for  $2^s$  compatible orders.*

*Proof.* Let  $<$  be a compatible order for which  $<$  is a  $p$ -proximity order. We prove that  $<'$  is a compatible order by induction on  $s$ . If  $s = 1$ , then the graph  $H$  is connected and there are only two partitions  $(L, R)$  and  $(R, L)$  and  $<'$  coincides with  $<$  or with its opposite, thus  $<'$  is compatible on  $\{p\} \cup \mathcal{K}_s^\oplus$ . For the same reason, for any connected component  $\mathcal{K}_i$  of the graph  $H$ , the restriction of  $<'$  on  $\{p\} \cup \mathcal{K}_i^\oplus$  is also compatible. Suppose by the induction hypothesis that the restriction of  $<'$  on the union  $\mathcal{K}_{\leq s-1}^\oplus$  of the first  $s - 1$  connected components is a compatible order. To prove that  $<'$  is compatible on  $X$ , pick any three points  $u, v, w$  such that  $u <' v <' w$ . We assert that  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$ . If  $\{u, v, w\} \subseteq \mathcal{K}_{\leq s-1}^\oplus$  or  $\{u, v, w\} \subseteq \mathcal{K}_s^\oplus$  the result follows by induction hypothesis or from the basic case.

First, let  $u, v \in \mathcal{K}_{\leq s-1}^\oplus$  and  $w \in \mathcal{K}_s^\oplus$ . By Lemma 6.13,  $\mathcal{K}_{\leq s-1}^\oplus$  is an mmodule, thus  $d(u, w) = d(v, w)$ . Since  $u < w$  and  $v < w$ ,  $w$  is not between  $p$  and  $u$  nor between  $p$  and  $v$  in  $<$ , hence  $w$  is not between  $u$  and  $v$  in  $<$ . Since  $<$  is a compatible order, independently of the position of  $w$ , we get  $d(u, v) \leq \max\{d(u, w), d(v, w)\} = d(u, w)$  and we are done. The analysis of case  $v, w \in \mathcal{K}_{\leq s-1}^\oplus$  and  $u \in \mathcal{K}_s^\oplus$  is analogous.

The case  $u, w \in \mathcal{K}_{\leq s-1}^\oplus$  and  $v \in \mathcal{K}_s^\oplus$  cannot happen, as  $\mathcal{K}_{\leq s-1}^\oplus$  is an interval of  $<'$ .

Next, let  $v \in \mathcal{K}_{\leq s-1}^\oplus$  and  $u, w \in \mathcal{K}_s^\oplus$ . Then because  $\mathcal{K}_{\leq s-1}^\oplus$  is an interval of  $<'$  containing  $p$ ,  $\text{side}(u) = L$  and  $\text{side}(w) = R$ . Since  $\mathcal{K}_{\leq s-1}^\oplus$  is an mmodule containing  $p$  and  $v$ , we deduce that  $d(p, u) = d(v, u)$  and  $d(p, w) = d(v, w)$ . Since  $<'$  is compatible on  $\{p\} \cup \mathcal{K}_s^\oplus$  and  $u <' p <' w$ , we get  $d(u, w) \geq \max\{d(u, p), d(p, w)\}$ . Combining these equalities and inequalities we obtain that  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$ .

Finally, suppose that  $w \in \mathcal{K}_{\leq s-1}^\oplus$  and  $u, v \in \mathcal{K}_s^\oplus$ . Since  $\mathcal{K}_{\leq s-1}^\oplus$  is an interval of  $<'$  containing  $p$ ,  $\text{side}(u) = \text{side}(v) = L$ . Since  $\mathcal{K}_{\leq s-1}^\oplus$  is an mmodule containing  $p$  and  $w$ , we have  $d(p, u) = d(w, u)$  and  $d(p, v) = d(w, v)$ .

Since  $<'$  is compatible on  $\{p\} \cup \mathcal{K}_s^\oplus$  and  $u <' v <' p$ , we have  $d(u, p) \geq \max\{d(u, v), d(v, p)\}$ . Combining these equalities and inequalities we obtain that  $d(u, w) \geq \max\{d(u, v), d(v, w)\}$ . The case when  $u \in \mathcal{K}_{\leq s-1}^\oplus$ ,  $v, w \in \mathcal{K}_s^\oplus$  is symmetrical. This concludes the proof of the first assertion.

To prove the second assertion it suffices to show how the compatible order  $<$ , for which  $<$  is a  $p$ -proximity order, can be obtained by our construction. Clearly, for each connected component  $\mathcal{K}_i$  of the graph  $H$ , the point  $m_i$  is located either to the left or to the right of  $p$ . Therefore  $\text{side}(m_i)$  can be fixed in this way and the bipartition  $(L_i, R_i)$  of  $\mathcal{K}_i^\oplus$  is uniquely determined by the value of  $\text{side}(m_i)$ . Moreover, this bipartition coincides with the bipartition of  $\mathcal{K}_i^\oplus$  with respect to the order  $<$ . By Lemma 6.7  $<$  coincides with the total order defined by the bipartition  $(L, R)$ , where  $L = \bigcup_{i=1}^s L_i$ ,  $R = \bigcup_{i=1}^s R_i$ .  $\square$

Summarizing the previous results, we obtain the following simple algorithm for computing a compatible order of  $(X, d)$  from a  $p$ -proximity order  $<$ :

1. Compute the graph  $G$  and the set  $\mathcal{C}$  of its connected components,
2. Compute the graph  $H$  and the set  $\Sigma = \{\mathcal{K}_1, \dots, \mathcal{K}_s\}$  of its connected components,
3. Compute the rightmost point  $m_i$  of each  $\mathcal{K}_i^\oplus$ ,  $i = 1, \dots, s$  with respect to  $<$  and arbitrarily fix  $\text{side}(m_i) \in \{L, R\}$ ,  $i = 1, \dots, s$ ,
4. Using  $\text{side}(m_i)$ , derive  $\text{side}(x)$  for all other points  $x \in \mathcal{K}_i^\oplus$  for  $i = 1, \dots, s$ ,
5. Return the total order  $<'$  on  $(X, d)$  in which the points of  $\{x \in X \setminus \{p\} : \text{side}(x) = L\}$  and of  $\{x \in X \setminus \{p\} : \text{side}(x) = R\}$  are ordered according to  $<$ .

The complexity of this algorithm is  $O(|X|^2)$  since the complexity of each of its steps is  $O(|X|^2)$ . The unique step requiring some explanation is the computation of  $H$ . For this, for each connected component  $\mathcal{C}_i$  of  $G$  we compute the minimum  $d_i$  and the maximum  $e_i$  according to  $<$ . Then we sort these segments  $[d_i, e_i]$  by the ending dates and sweep the sorted list to return the pairs of intersecting segments  $[d_i, e_i]$  and  $[d_j, e_j]$ . This corresponds to the edges of  $H$  and all this can be done in  $O(|X|^2)$ .

In fact, to compute the sides of points and the resulting compatible order, we do not need to explicitly compute the graphs  $G$  and  $H$ . This leads us to Algorithm 6.2. Actually, this algorithm simultaneously constructs the connected components  $\mathcal{K}_i$  of  $H$  (without building all the components of the graph  $G$ ) and assigns the points of each  $\mathcal{K}_i^\oplus$  to the correct set  $L$  or  $R$ .

---

**Algorithm 6.2** sortByBipartition( $p, X$ )

---

**Input:** A Robinson space  $(X, d)$  ( $d$  is implicit), a point  $p \in X$ ,  $X \setminus \{p\}$  is given in  $p$ -proximity order.

**Output:**  $X$  in a compatible order.

```

1: let  $L = [], R = [], Undecided = \text{reverse}(X \setminus \{p\})$ 
2: for  $q \in X \setminus \{p\}$  in decreasing order do
3:   if  $q \in Undecided$  then
4:     choose arbitrarily: either  $R \leftarrow q \cdot R$  or  $L \leftarrow q \cdot L$ 
5:      $Undecided \leftarrow Undecided \setminus \{q\}$ 
6:   let  $Skipped = []$ 
7:   for  $x \in Undecided$  from first to last do
8:     remove  $x$  from  $Undecided$ 
9:     if  $d(x, q) = d(p, q)$  then
10:       $Skipped \leftarrow x \cdot Skipped$ 
11:    else
12:      if  $(d(x, q) < d(p, q) \text{ and } q \in L) \text{ or } (d(x, q) > d(p, q) \text{ and } q \in R)$  then
13:         $L \leftarrow x \cdot L$ 
14:         $R \leftarrow Skipped \uplus R$ 
15:      else
16:         $R \leftarrow x \cdot R$ 
17:         $L \leftarrow Skipped \uplus L$ 
18:       $Skipped \leftarrow []$ 
19:    $Undecided \leftarrow \text{reverse}(Skipped)$ 
20: return  $\text{reverse}(L) \uplus [p] \uplus R$ 

```

---

PROPOSITION 6.15. *Given a Robinson space  $(X, d)$  with  $|X| = n$ , a point  $p \in X$ , and a  $p$ -proximity*

order  $<$  on  $X \setminus \{p\}$ , **Algorithm 6.2** returns  $X$  sorted along a compatible order in  $O(n^2)$  time.

*Proof.* We first prove the correctness of **Algorithm 6.2**. In the proof, we will use the following invariants of the algorithm that hold at the end of each iteration of any loop. These invariants can be readily checked.

- (i) The list *Undecided* is always sorted in decreasing  $p$ -proximity order,
- (ii) *Skipped*,  $L$  and  $R$  are sorted in increasing  $p$ -proximity order.
- (iii) each element is in exactly one of the four lists *Undecided*, *Skipped*,  $L$  and  $R$ .
- (iv) for any  $x \in \text{Undecided}$ ,  $y \in \text{Skipped}$  and  $z \in L \cup R$ , we have  $x < y < z$ .

Obviously  $L$  and  $R$  contain the elements of each side, while *Undecided* and *Skipped* contains elements whose side is not determined yet.

Notice that the first element  $q$  for which Line 4 is executed is  $m_s$ . Consider the iterations of Loop 2–19 starting from the first one and as long as Line 4 is not executed again. We claim that at the end of these iterations, *Undecided* contains exactly  $\mathcal{K}_{\leq s-1}^\oplus$ , while  $L \cup R$  contains  $\mathcal{K}_s^\oplus$ . We denote  $S \subseteq X \setminus \{p\}$  the set of elements in  $L \cup R$  at that time. First we prove that if a component  $C$  of  $G$  intersects  $S$ , then  $C \subseteq S$ , in particular  $C(m_s) \subseteq S$ . Suppose not, then there is an edge  $uv$  in  $G$  with  $u, v \in C$ ,  $u \notin S$  and  $v \in S$ , and by invariant (iv)  $u < v$ . But then during the Loop 2–19 for  $q = v$  and Loop 7–18 for  $x = u$ ,  $u$  should have been decided on Lines 12–18, contradiction. Then we prove that if  $C, C'$  is a tangled pair with  $u, v \in C$ ,  $w \in C'$ ,  $u < w < v$ ,  $C \subseteq S$ , then  $C' \subseteq S$ . This follows immediately from invariant (iv). Finally, as  $G$  has no edge between  $\mathcal{K}_{\leq s-1}^\oplus$  and  $\mathcal{K}_s^\oplus$ , when  $x$  is an element of  $\mathcal{K}_{\leq s-1}^\oplus$ ,  $d(x, q) = d(p, q)$  during these iterations, hence  $x$  is skipped (Line 10), proving our claim.

From this we proceed by induction on  $s$ . It only remains to prove that each element is correctly assigned to its side. On Line 4, it follows from the fact that  $q = m_i$ , hence we can choose arbitrarily by **Proposition 6.14**. On Lines 13 and 16, it follows from **Lemma 6.8**. On Lines 14 and 17, it follows from **Lemma 6.11**. This proves the correction of **Algorithm 6.2**.

The complexity of **Algorithm 6.2** is easily derived, since each loop iterates at most  $n$  times and by observing that the complexity of appending *Skipped* is amortized over the insertions into *Skipped*.  $\square$

Consequently, we obtain the following result:

**PROPOSITION 6.16.** *Let  $(X, d)$  be  $p$ -trivial or flat Robinson space on  $n$  points. Then a compatible order on  $X$  can be computed in  $O(n^2 + T)$  time, where  $T$  is the total time used by the refine procedure. Analogously, if  $(C_p^*, d^*)$  is an extended quotient of a Robinson space  $(X, d)$  with  $k$  copoints, then a compatible order on  $C_p^*$  can be computed in  $O(k^2 + T)$ .*

*Proof.* If  $(X, d)$  is  $p$ -trivial, then the result follows from **Propositions 6.5** and **6.15**. Now suppose that  $(X, d)$  is flat and let  $p$  be a diametral point of  $(X, d)$ . By **Proposition 4.1**,  $(X, d)$  is  $p$ -trivial, thus we can apply the previous case. Finally, if  $(C_p^*, d^*)$  is an extended quotient of a Robinson space  $(X, d)$ , then  $C_p^*$  contains at most  $2k$  points. Consequently the result follows from **Propositions 6.6** and **6.15** and **Lemma 6.7**.  $\square$

**The running example.** We illustrate **Algorithm 6.2** on some subspace of the running example. Consider the 1-proximity order

$$1 < 17 < 9 < 10 < 6 < 4 < 13 < 7 < 19$$

for which we want to find a compatible order, with dissimilarities given in **Figure 6.1** (left).

First, we consider  $q = 19$ , with  $d(p, q) = 10$ , on Line 4, say we choose  $19 \in L$ . Then for  $x = 7$ ,  $d(q, x) = 11 > d(p, q) = 10$ , hence  $7 \in R$ . Similarly, for  $x = 13$  and  $x = 4$ , the algorithm decides that  $13 \in L$ ,  $4 \in L$ . Then all remaining elements are skipped.

Then for  $q \in \{7, 13, 4\}$  all remaining elements are again skipped. When  $q = 6$ , we get to choose arbitrarily on Line 4 that  $6 \in R$ . Then  $d(p, q) = 8$ . For  $x = 10$ ,  $d(q, x) = 7 < d(p, q)$  implies  $10 \in R$ , for  $x = 9$ ,  $d(q, x) = 9 > d(p, q)$  implies  $9 \in L$ , and for  $x = 17$ ,  $d(q, x) = 8 = d(p, q)$ , 17 is skipped.

Then for  $q = 10$ ,  $d(p, q) = 8 > d(17, q)$ , hence  $17 \in R$ . The algorithm returns the compatible order

$$19 < 13 < 4 < 9 < 1 < 17 < 10 < 6 < 7,$$

whose matrix is given in **Figure 6.1** (right).

**7. A divide-and-conquer algorithm.** In this section, we describe the divide-and-conquer algorithm for recognizing Robinson spaces, prove its correctness and establish its running time.

|    |   |    |   |    |   |   |    |   |    |    |    |    |   |    |    |    |    |    |    |
|----|---|----|---|----|---|---|----|---|----|----|----|----|---|----|----|----|----|----|----|
|    | 1 | 17 | 9 | 10 | 6 | 4 | 13 | 7 | 19 |    | 19 | 13 | 4 | 9  | 1  | 17 | 10 | 6  | 7  |
| 1  | 0 | 4  | 4 | 8  | 8 | 9 | 9  | 9 | 10 | 19 | 0  | 5  | 8 | 10 | 10 | 10 | 10 | 10 | 11 |
| 17 |   | 0  | 6 | 7  | 8 | 9 | 9  | 9 | 10 | 13 |    | 0  | 6 | 9  | 9  | 9  | 9  | 9  | 9  |
| 9  |   |    | 0 | 8  | 9 | 9 | 9  | 9 | 10 | 4  |    |    | 0 | 9  | 9  | 9  | 9  | 9  | 9  |
| 10 |   |    |   | 0  | 7 | 9 | 9  | 9 | 10 | 9  |    |    |   | 0  | 4  | 6  | 8  | 9  | 9  |
| 6  |   |    |   |    | 0 | 9 | 9  | 9 | 10 | 1  |    |    |   |    | 0  | 4  | 8  | 8  | 9  |
| 4  |   |    |   |    |   | 0 | 6  | 9 | 8  | 17 |    |    |   |    |    | 0  | 7  | 8  | 9  |
| 13 |   |    |   |    |   |   | 0  | 9 | 5  | 10 |    |    |   |    |    |    | 0  | 7  | 9  |
| 7  |   |    |   |    |   |   |    | 0 | 11 | 6  |    |    |   |    |    |    |    | 0  | 9  |
| 19 |   |    |   |    |   |   |    |   | 0  | 7  |    |    |   |    |    |    |    |    | 0  |

FIG. 6.1. An extended quotient space from the running example, given in 1-proximity order (left) and compatible order (right).

**7.1. The algorithm.** The results of Sections 3, 5 and 6 (namely, Proposition 5.6, Proposition 5.9, Proposition 6.16 and Algorithm 6.1, Algorithm 4.1, Algorithm 6.2) lead to the following algorithm for computing a compatible order of a Robinson space  $(X, d)$ :

1. Compute a copoint partition  $\mathcal{C}_p$  of  $(X, d)$  using Algorithm 6.1,
2. Recursively find a compatible order  $<_i$  for each copoint  $C_i$  of  $\mathcal{C}_p$ ,
3. Classify the copoints of  $\mathcal{C}_p$  into separable, tight, and non-separable, separate the separable copoints using Algorithm 4.1, and construct the extended quotient  $(\mathcal{C}_p^*, d^*)$  of  $(X, d)$ ,
4. Compute a  $p$ -proximity order  $<$  for the extended quotient  $(\mathcal{C}_p^*, d^*)$  using Algorithm 6.2,
5. Build a compatible order  $<^*$  for  $(\mathcal{C}_p^*, d^*)$  using  $<$ ,
6. Merge the compatible order  $<^*$  on  $\mathcal{C}_p^*$  with the compatible orders  $<_i$  on the copoints  $C_i$  of  $\mathcal{C}_p$  to get a total order  $<$  on  $X$ , using Proposition 5.9,
7. If  $<$  is not a compatible order of  $(X, d)$ , then return “not Robinson”, otherwise return  $<$ .

The pseudo-code of the algorithm is Algorithm 7.1.

---

**Algorithm 7.1** findCompatibleOrder( $X$ )

---

**Input:** a Robinson space  $(X, d)$  ( $d$  is implicit).

**Output:** a compatible order for  $X$  (as a sorted list).

```

1: if  $X$  is empty then
2:   return []
3: let  $p \in X$ ,  $X' = X \setminus \{p\}$ 
4: let  $[C_1, \dots, C_k] = \text{recursiveRefine}(p, [p], X', [])$ 
5: let  $\text{representedCopoints} = []$ 
6: for  $i \in \{1, \dots, k\}$  in decreasing order do
7:   let  $C'_i = \text{findCompatibleOrder}(C_i)$ 
8:    $\text{representedCopoints} \leftarrow \text{separateIfSeparable}(p, C'_i) \uplus \text{representedCopoints}$ 
9: let  $[(x_1, T_1), \dots, (x_{k'}, T_{k'})] = \text{representedCopoints}$ 
10: let  $[x_{\sigma(1)}, \dots, p, \dots, x_{\sigma(k')}] = \text{sortByBipartition}(p, [x_1, \dots, x_{k'}])$ 
11: return concatenate( $T_{\sigma(1)}, \dots, [p], \dots, T_{\sigma(k')}$ )

```

---

To represent the extended quotient  $(\mathcal{C}_p^*, d^*)$ , we select a set of representatives of tight, non-separable, and halved copoints: the point  $p$ , one representative  $x_i$  for each tight or non-separable copoint  $C_i$ , and a diametral pair  $(x'_i, x''_i)$  for each separable copoint.

**7.2. Complexity and correctness of the algorithm.** The correctness and the complexity of Algorithm 6.2 (*sortByBipartition*), Algorithm 4.1 (*separateIfSeparable*) and Algorithm 6.1 (*recursiveRefine*) was established in Subsections 4.2, 6.1 and 6.3. For Algorithm 6.1, the complexity was established without counting the calls of Algorithm 2.2 (*refine*). This will be done here.

We will use the following auxiliary result:

LEMMA 7.1. *If  $T : \mathbb{N} \rightarrow \mathbb{N}$  satisfies the recurrence relation  $T(n) \leq \sum_{i=1}^k T(n_i) + n \log k$ , for all partitions  $\sum_{i=1}^k n_i = n$  of  $n$  into  $k \geq 2$  positive integers, then  $T(n) = O(n^2)$ .*

*Proof.* By convexity of the function  $\sum_{i=1}^k x_i^2$ , the maximum of  $\sum_{i=1}^k n_i^2$  over all partitions of  $n$  in  $k$  parts is attained by a partition with one class with  $n - k + 1$  points and  $k - 1$  singletons, and has value  $(n - k + 1)^2 + (k - 1)$ . Assume that for all  $p < n$ ,  $T(p) \leq \alpha p^2$  for some  $\alpha \geq 1$ . Then

$$\begin{aligned} T(n) &\leq \sum_{i=1}^k \alpha n_i^2 + n \log k \\ &\leq \alpha(n - k + 1)^2 + \alpha(k - 1) + n \log k \\ &= \alpha n^2 - \alpha(k - 1)(2n - k) + n \log k \\ &\leq \alpha n^2 - n(\alpha(k - 1) - \log k) \end{aligned}$$

where the last inequality follows from  $2n - k \geq n$ . It suffices to prove that  $\alpha(k - 1) - \log k$  is nonnegative, which is true because  $\alpha \geq 1$  and  $k \geq 2$ .  $\square$

We continue with the main result of the paper.

**THEOREM 7.2.** *Algorithm 7.1 computes a compatible order of a Robinson space  $(X, d)$  in  $O(n^2)$  time.*

*Proof.* The correction follows from Proposition 5.9 that proves that a compatible order on  $(X, d)$  can be built by composing a compatible order on each copoint or halved copoint with a compatible order on the extended quotient space (which exists by Proposition 5.9). By Proposition 4.5 and by induction, each  $T_i$  is a tight or non-separable copoint or a halved copoint in increasing compatible order, with representative  $x_i$ , and by Proposition 6.15,  $\sigma$  sorts the representatives  $(x_i)_{i \in \{1, \dots, k\}}$  and  $p$  in a compatible order, so that Proposition 5.9 applies.

We analyze the complexity of Algorithm 7.1 by counting separately the number of operations done in the procedures *refine*, *recursiveRefine* and *sortByBipartition*. All the other operations can be done in linear-time at each level of recursion, thus in  $O(|X|^2)$  times in total.

- *recursiveRefine* contributes  $O(|X|^2)$  in the total complexity; indeed, applying Lemma 6.4, the first call takes  $\alpha(|X|^2 - \sum_{i=1}^k |C_i|^2)$  (for some constant  $\alpha$ ), while the cost of *recursiveRefine* in the recursive calls are at most  $\alpha \sum_{i=1}^k |C_i|^2$  by induction, summing to  $\alpha|X|^2$ .
- *refine* contributes  $O(|X|^2)$  in the total complexity, because it follows the recurrence relation described in Lemma 7.1.
- *sortByBipartition* contributes  $O(|X|^2)$  in the total complexity; indeed, considering the recursion tree of calls to *findCompatibleOrder*, one can see that each call to *sortByBipartition* uses  $O(k^2)$  operations where  $k$  is the arity of the corresponding node. Hence the contribution of *sortByBipartition* is of the form  $\beta \sum_{i=1}^l k_i^2$  where  $l$  is the number of nodes and  $k_i$  the arity of the  $k_i$ th node. But, because each node, inner or leaf, can be associated to a unique element in  $X$ ,  $\sum_{i=1}^l k_i = |X| - 1$ , implying that  $\beta \sum_{i=1}^l k_i^2 \leq \beta|X|^2$  by convexity.

Summing up all the contributions, we get that *findCompatibleOrder* runs in time  $O(|X|^2)$ .  $\square$

**REMARK 2.** *Algorithm 7.1 can be transformed into a recognition algorithm by simply testing in  $O(|X|^2)$  time if the returned sorted list is a compatible order on  $(X, d)$ . If this is not the case, from the results of previous sections it follows that  $(X, d)$  is not a Robinson space.*

**REMARK 3.** *If  $(X, d)$  is  $p$ -trivial, then all copoints  $C_i$  have size 1, thus in this case the Algorithm 7.1 is no longer recursively applied to the copoints. In particular, this is the case if  $(X, d)$  is a flat Robinson space and  $p$  is diametral, since by Proposition 4.1  $(X, d)$  is then  $p$ -trivial.*

**The running example.** We conclude this section by running *findCompatibleOrder* on the running example. On Line 3, we chose  $p = 1$  and then on Line 4 we build the ordered copoint partition

$$C_1 = [\{1\}, \{17\}, \{9\}, \{10\}, \{6\}, \{3, 4, 8, 16, 18\}, \{11, 13, 14\}, \{7\}, \{2, 5, 12, 15, 19\}]$$

by using *recursiveRefine* (see Subsection 6.1). It returns the following orders of non-trivial copoints:

- $4 < 3 < 18 < 8 < 16$ ,
- $13 < 14 < 11$ ,



-  $19 < 5 < 15 < 2 < 12$ ,

None of these copoints is separable, hence we obtain the points 17, 9, 10, 6, 4, 13, 7, 19 as representatives of the quotient space, on which we call *sortByBipartition* with  $p = 1$  on Line 10. As seen at the end of Subsection 6.3, this outputs the compatible order  $19 < 13 < 4 < 9 < 1 < 17 < 10 < 6 < 7$ . Then on Line 11 concatenating all the copoints in the same order as their representatives, we get the compatible order:

$$19 < 5 < 15 < 2 < 12 < 13 < 14 < 11 < 4 < 3 < 18 < 8 < 16 < 9 < 1 < 17 < 10 < 6 < 7.$$

We check that this order is compatible by verifying the monotonicity of rows and columns of Figure 7.1.

To illustrate separable copoints, consider the recursive call to the copoint  $\{2, 5, 12, 15, 19\}$ . With  $p = 2$  as pivot, the ordered copoint partition is  $[\{2\}, \{15\}, \{5, 12, 19\}]$ . We determine recursively that  $\{5, 12, 19\}$  has  $19 < 5 < 12$  as a compatible order. As seen after Definition 5.2,  $\{5, 12, 19\}$  is a separable copoint with halved copoints  $\{5, 19\}$  and  $\{12\}$ , thus we can take  $[2, 15, 19, 12]$  as extended quotient space. Then *sortByBipartition* returns the order  $19 < 15 < 2 < 12$ , from which we get the compatible order  $19 < 5 < 15 < 2 < 12$ .

|    | 19 | 5 | 15 | 2 | 12 | 13 | 14 | 11 | 4 | 3 | 18 | 8 | 16 | 9  | 1  | 17 | 10 | 6  | 7  |
|----|----|---|----|---|----|----|----|----|---|---|----|---|----|----|----|----|----|----|----|
| 19 | 0  | 1 | 2  | 2 | 4  | 5  | 5  | 5  | 8 | 8 | 8  | 8 | 8  | 10 | 10 | 10 | 10 | 10 | 11 |
| 5  |    | 0 | 2  | 2 | 3  | 5  | 5  | 5  | 8 | 8 | 8  | 8 | 8  | 10 | 10 | 10 | 10 | 10 | 11 |
| 15 |    |   | 0  | 1 | 2  | 5  | 5  | 5  | 8 | 8 | 8  | 8 | 8  | 10 | 10 | 10 | 10 | 10 | 11 |
| 2  |    |   |    | 0 | 2  | 5  | 5  | 5  | 8 | 8 | 8  | 8 | 8  | 10 | 10 | 10 | 10 | 10 | 11 |
| 12 |    |   |    |   | 0  | 2  | 5  | 5  | 8 | 8 | 8  | 8 | 8  | 10 | 10 | 10 | 10 | 10 | 11 |
| 13 |    |   |    |   |    | 0  | 1  | 1  | 6 | 6 | 6  | 6 | 6  | 9  | 9  | 9  | 9  | 9  | 9  |
| 14 |    |   |    |   |    |    | 0  | 1  | 6 | 6 | 6  | 6 | 6  | 9  | 9  | 9  | 9  | 9  | 9  |
| 11 |    |   |    |   |    |    |    | 0  | 6 | 6 | 6  | 6 | 6  | 9  | 9  | 9  | 9  | 9  | 9  |
| 4  |    |   |    |   |    |    |    |    | 0 | 1 | 2  | 2 | 3  | 9  | 9  | 9  | 9  | 9  | 9  |
| 3  |    |   |    |   |    |    |    |    |   | 0 | 2  | 2 | 2  | 9  | 9  | 9  | 9  | 9  | 9  |
| 18 |    |   |    |   |    |    |    |    |   |   | 0  | 2 | 2  | 9  | 9  | 9  | 9  | 9  | 9  |
| 8  |    |   |    |   |    |    |    |    |   |   |    | 0 | 2  | 9  | 9  | 9  | 9  | 9  | 9  |
| 16 |    |   |    |   |    |    |    |    |   |   |    |   | 0  | 9  | 9  | 9  | 9  | 9  | 9  |
| 9  |    |   |    |   |    |    |    |    |   |   |    |   |    | 0  | 4  | 6  | 8  | 9  | 9  |
| 1  |    |   |    |   |    |    |    |    |   |   |    |   |    |    | 0  | 4  | 8  | 8  | 9  |
| 17 |    |   |    |   |    |    |    |    |   |   |    |   |    |    |    | 0  | 7  | 8  | 9  |
| 10 |    |   |    |   |    |    |    |    |   |   |    |   |    |    |    |    | 0  | 7  | 9  |
| 6  |    |   |    |   |    |    |    |    |   |   |    |   |    |    |    |    |    | 0  | 9  |
| 7  |    |   |    |   |    |    |    |    |   |   |    |   |    |    |    |    |    |    | 0  |

FIG. 7.1. The distance matrix  $D_C$  of the Robinson space of Figure 2.1 with the entries ordered along the compatible order 19, 5, 15, 2, 12, 13, 14, 11, 4, 3, 18, 8, 16, 9, 1, 17, 10, 6, 7.

**8. Conclusion.** In this paper, we investigated the structure of mmodules and copoint partitions in general dissimilarity spaces, and, more particularly, in Robinson spaces. We proved that the mmodules of any dissimilarity space can be represented using the mmodule-tree and that the maximal mmodules not containing a given point form a partition (which we called a copoint partition). The copoint partition leads to the quotient dissimilarity space, which reflect the large scale structure of the dissimilarity space. In Robinson spaces, we proved that the mmodules and the copoint partitions satisfy stronger properties. We classified the copoints into separable, non-separable, and tight and proved that in any compatible order each separable copoint define two intervals, each non-separable copoint is a single interval, and that there exists a compatible order in which all tight copoints are intervals. After partitioning each separable copoint into two parts, we obtain the extended quotient space. We prove that any such extended quotient admits a  $p$ -proximity order and we show how to compute it efficiently.

Based on all these results and notions, we presented a divide-and-conquer algorithm for recognizing Robinson matrices in optimal  $O(n^2)$  time. Our algorithm first computes a copoint partition, then recursively computes a compatible order of each copoint of the partition, classifies the copoints and partitions the separable copoints, constructs a  $p$ -proximity order of the extended quotient. Finally, from the  $p$ -proximity order and the compatible orders of its copoints, it derives a compatible order of the whole space. Our algorithm does not partition the tight copoints, although there may exist compatible orders in which some tight copoints are partitioned. Thus, our algorithm does not return all compatible orders. In the companion paper [7], we also establish a correspondence between the mmodule-tree of a Robinson dissimilarity and its PQ-tree. PQ-trees are used to encode all compatible orders of a Robinson space and using one such compatible order (say, computed by our algorithm), it can be shown that one can construct the PQ-tree.

Mmodule-trees, the copoints partitions, and their quotient spaces may be viewed as generic ingredients when investigating general dissimilarity spaces and may be useful for the recognition of other classes of dissimilarities, in particular of tree-Robinson and circular-Robinson dissimilarities. For example, the approach via mmodules was one of the starting points of our optimal algorithm for strict circular seriation in [8]. One can also easily characterize ultrametrics by their mmodule-tree.

As we already mentioned in the introduction, the first recognition algorithm of Robinson spaces running in optimal  $O(n^2)$  time was presented by Préa and Fortin [33]. It first constructs (by the Booth and Lueker’s algorithm [5]) a PQ-tree representing a super set of the compatible permutations (if the dissimilarity is Robinson). In a second stage, the PQ-tree is refined, in  $O(n^2)$  time, in a such way that the set of represented permutations coincides with the set of the compatible ones. The Booth and Lueker’s algorithm is renowned to be tricky to be efficiently implemented, and the second step is even more evolved. Due to this, even if optimal, the algorithm of [33] is not simple.

Our optimal recognition algorithm is simple and was relatively easy to implement in OCaml [9], as it should also be in any mainstream programming language. Since it uses only basic data structures, it can be casted as practical. The program solves seemingly hard instances on 1000 points in half a second, and instances on 10000 points in less than a minute on a standard laptop (the algorithm of [33] was implemented by Préa and does not show the same practical performances). Among the different random generators we used to evaluate our program, the hardest instances were obtained by shuffling Robinson Toeplitz matrices with coefficients in  $\{0, 1, 2\}$ .

**Acknowledgement.** We would like to acknowledge the referees for their careful reading of the original and the revised versions of the manuscript, and useful suggestions and comments. This research was supported in part by ANR project DISTANCIA (ANR-17-CE40-0015) and has received funding from Excellence Initiative of Aix-Marseille - A\*MIDEX (Archimedes Institute AMX-19-IET-009), a French “Investissements d’Avenir” Program.

## REFERENCES

- [1] J. ARACENA AND C. THRIVES CARO, *On the recognition of strong-Robinsonian incomplete matrices*, arXiv:2101.03033, (2021).
- [2] S. ARMSTRONG, C. GUZMÁN, AND C. A. SING LONG, *An optimal algorithm for strict circular seriation*, SIAM J. Math. Data Science, 3 (2021), pp. 1223–1250.
- [3] J. E. ATKINS, E. G. BOMAN, AND B. HENDRICKSON, *A spectral algorithm for seriation and the consecutive ones problem*, SIAM J. Comput., 28 (1998), pp. 297–310.
- [4] J.-P. BARTHÉLEMY AND F. BRUCKER, *NP-hard approximation problems in overlapping clustering*, J. Classif., 18 (2001), pp. 159–183.
- [5] K. S. BOOTH AND G. S. LUEKER, *Testing for the consecutive ones property, interval graphs, and graph planarity using PQ-tree algorithms*, J. Comput. Syst. Sci., 13 (1976), pp. 335–379.
- [6] M. J. BRUSCO, *A branch-and-bound algorithm for fitting anti-Robinson structures to symmetric dissimilarity matrices*, Psychometrika, 67 (2002), pp. 459–471.
- [7] M. CARMONA, V. CHEPOI, G. NAVES, AND P. PRÉA, *Modules and PQ-trees in Robinson spaces*, 2022. in preparation.
- [8] M. CARMONA, V. CHEPOI, G. NAVES, AND P. PRÉA, *A simple and optimal algorithm for strict circular seriation*, SIAM J. Math. Data Science, 5 (2023), pp. 201–221.
- [9] M. CARMONA, V. CHEPOI, G. NAVES, AND P. PRÉA, *Implementations of recognition algorithms for Robinson dissimilarities*. <https://gitlab.lis-lab.fr/guyslain.naves/dissiml>, 2023.
- [10] M. CHEIN, M. HABIB, AND M. MAURER, *Partitive hypergraphs*, Discrete Math., 37 (1981), pp. 35–50.
- [11] V. CHEPOI AND B. FICHET, *Recognition of Robinsonian dissimilarities*, J. Classif., 14 (1997), pp. 311–325.
- [12] V. CHEPOI, B. FICHET, AND M. SESTON, *Seriation in the presence of errors: NP-hardness of  $l_\infty$ -fitting Robinson structures to dissimilarity matrices*, J. Classif., 26 (2009), pp. 279–296.
- [13] V. CHEPOI AND M. SESTON, *Seriation in the presence of errors: A factor 16 approximation algorithm for  $l_\infty$ -fitting Robinson structures to distances*, Algorithmica, 59 (2011), pp. 521–568.
- [14] F. CRITCHLEY AND B. FICHET, *The partial order by inclusion of the principal classes of dissimilarity on a finite set, and some of their basic properties*, in Classification and Dissimilarity Analysis, Springer, 1994, pp. 5–65.
- [15] E. DIDAY, *Orders and overlapping clusters in pyramids.*, in Multidimensional Data Analysis, J. e. a. de Leeuw, ed., DSWO Press, Leiden, 1986.
- [16] C. DURAND AND B. FICHET, *One-to-one correspondences in pyramidal representation: A unified approach.*, in Conf. of IFCS, 1987, pp. 85–90.
- [17] A. EHRENFUCHT, H. N. GABOW, R. M. MCCONNELL, AND S. SULLIVAN, *An  $O(n^2)$  divide-and-conquer algorithm for the prime tree decomposition of two-structures and modular decomposition of graphs*, J. Algorithms, 16 (1994), pp. 283–294.
- [18] A. EHRENFUCHT AND G. ROZENBERG, *Theory of 2-structures, part I: Clans, basic subclasses, and morphisms*, Theor.

- Comput. Sci., 70 (1990), pp. 277–303.
- [19] A. EHRENFUCHT AND G. ROZENBERG, *Theory of 2-structures, part II: Representation through labeled tree families*, Theor. Comput. Sci., 70 (1990), pp. 305–342.
  - [20] F. FOGEL, A. D’ASPREMONT, AND M. VOJNOVIC, *Serialrank: Spectral ranking using seriation*, NeuroIPS, 27 (2014).
  - [21] M. GHANDEHARI AND J. JANSSEN, *An optimization parameter for seriation of noisy data*, SIAM J. Discr. Math., 33 (2019), pp. 712–730.
  - [22] M. HABIB AND C. PAUL, *A survey of the algorithmic aspects of modular decomposition*, Comput. Sci. Review, 4 (2010), pp. 41–59.
  - [23] L. HUBERT, *Some applications of graph theory and related non-metric techniques to problems of approximate seriation: The case of symmetric proximity measures*, British J. Math. Stat. Psychol., 27 (1974), pp. 133–153.
  - [24] L. HUBERT, P. ARABIE, AND J. MEULMAN, *Graph-theoretic representations for proximity matrices through strongly-anti-Robinson or circular strongly-anti-Robinson matrices*, Psychometrika, 63 (1998), pp. 341–358.
  - [25] L. HUBERT, P. ARABIE, AND J. MEULMAN, *The structural representation of proximity matrices with MATLAB*, ASA-SIAM Series on Statistics and Applied Probability, SIAM, 2006.
  - [26] Y. ISSARTEL, C. GIRAUD, AND N. VERZELEN, *Optimal embedding on the sphere in non-parametric latent space models*, 2021.
  - [27] R. E. JAMISON, *Copoints in antimatroids*, Congr. Numer., 29 (1980), pp. 535–544.
  - [28] M. LAURENT AND M. SEMINAROTI, *A Lex-BFS-based recognition algorithm for Robinsonian matrices*, Discr. Appl. Math., 222 (2017), pp. 151–165.
  - [29] M. LAURENT AND M. SEMINAROTI, *Similarity-first search: a new algorithm with application to Robinsonian matrix recognition*, SIAM J. Discr. Math., 31 (2017), pp. 1765–1800.
  - [30] M. LAURENT, M. SEMINAROTI, AND S. TANIGAWA, *A structural characterization for certifying Robinsonian matrices*, Electr. J. Combin., 24 (2017), p. P2.21.
  - [31] B. MIRKIN AND S. RODIN, *Graphs and Genes*, Springer, 1984.
  - [32] C. OKASAKI, *Purely Functional Data Structures*, Cambridge University Press, 1999.
  - [33] P. PRÉA AND D. FORTIN, *An optimal algorithm to recognize Robinsonian dissimilarities*, J. Classif., 31 (2014), pp. 351–385.
  - [34] A. RECANATI, T. KERDREUX, AND A. D’ASPREMONT, *Reconstructing latent orderings by spectral clustering*, arXiv preprint arXiv:1807.07122, (2018).
  - [35] W. S. ROBINSON, *A method for chronologically ordering archaeological deposits*, Amer. Antiq., 16 (1951), pp. 293–301.
  - [36] M. SESTON, *Dissimilarités de Robinson: algorithmes de reconnaissance et d’approximation*, PhD thesis, Université de la Méditerranée, Aix-Marseille 2, 2008.
  - [37] M. SESTON, *A simple algorithm to recognize Robinsonian dissimilarities*, COMPSTAT 2008, (2008), pp. 241–248.
  - [38] M. L. VAN DE VEL, *Theory of Convex Structures*, Elsevier, 1993.